

EGL 1.0 Fonksiyonları

Bu dosya, ayrı EGL fonksiyon belgelerinin tipik bir EGL yaşam döngüsüne göre sıralanmış birleşimidir.

İçindekiler

1. `eglGetDisplay`
2. `eglInitialize`
3. `eglGetConfigs`
4. `eglChooseConfig`
5. `eglGetConfigAttrib`
6. `eglCreateWindowSurface`
7. `eglCreateContext`
8. `eglMakeCurrent`
9. `eglGetCurrentDisplay`
10. `eglGetCurrentContext`
11. `eglSwapBuffers`
12. `eglDestroyContext`
13. `eglDestroySurface`
14. `eglTerminate`
15. `eglGetError`

EGL 1.0: `eglGetDisplay`

```
EGLDisplay eglGetDisplay(EGLNativeDisplayType display_id);
```

`eglGetDisplay` , native görüntüleme sistemine ait bir display tanımlayıcısını EGL tarafından kullanılabilen bir `EGLDisplay` handle'ı ile ilişkilendirmek için kullanılır.

Kısa özet:

- Girdi: `EGLNativeDisplayType`

- Çıktı: `EGLDisplay`
- Başarısızlık değeri: `EGL_NO_DISPLAY`
- `EGL_DEFAULT_DISPLAY` , varsayılan native display'i istemek için kullanılır.
- `EGLDisplay` , fiziksel monitörün kendisi değil, EGL'nin kullandığı opaque handle'dır.
- `eglGetDisplay` display'i initialize etmez; bunun için `eglInitialize` gerekir.

Mental Model

```
Native display
  |
  v
eglGetDisplay
  |
  v
EGLDisplay handle
  |
  v
eglInitialize
```

`eglGetDisplay` native platform ile EGL arasındaki ilk bağlantı noktalarından biridir.

Parametreler

`display_id`

`display_id` , hangi native display'in EGL tarafında temsil edileceğini belirtir.

`EGLNativeDisplayType` platforma bağımlıdır. Ubuntu header dosyalarında farklı platformlar için örneğin şu tanımlar bulunur:

```
typedef struct wl_display *EGLNativeDisplayType;
typedef Display *EGLNativeDisplayType;
typedef void *EGLNativeDisplayType;
typedef int EGLNativeDisplayType;
```

Aynı anda bunların hepsi aktif değildir; platform header'ları uygun tanımları seçer.

Deney sisteminde `EGL_DEFAULT_DISPLAY` şu şekilde tanımlanmıştır:

```
#define EGL_DEFAULT_DISPLAY EGL_CAST(EGLNativeDisplayType,0)
```

display_id değeri	Sonuç / anlam
EGL_DEFAULT_DISPLAY	Varsayılan native display için EGLDisplay istenir.
Geçerli Wayland wl_display *	Bu explicit native bağlantı için EGLDisplay istenir.
Aynı EGL_DEFAULT_DISPLAY tekrar	Test edilen Mesa implementasyonunda aynı handle döndü.
Farklı native Wayland bağlantıları	Test edilen Mesa implementasyonunda farklı EGLDisplay handle'ları döndü.

EGLDisplay Nedir?

EGLDisplay , uygulamanın iç yapısını yorumlamaması gereken bir EGL handle'dır.

Örnek:

```
0x629b25a1cdf0
```

Bu değer:

- çözünürlük değildir,
- ekran numarası değildir,
- GPU numarası değildir,
- EGL sürümü değildir.

Uygulama bu handle'ı daha sonraki EGL çağrılarına geçirir:

```
eglInitialize(display, &major, &minor);
```

eglInitialize ayrıntıları için ilgili bölüme bakınız.

EGL_DEFAULT_DISPLAY

En temel kullanım:

```
EGLDisplay dpy = eglGetDisplay(EGL_DEFAULT_DISPLAY);

if (dpy == EGL_NO_DISPLAY) {
    /* Display elde edilemedi. */
}
```

Bu kullanımda native display bağlantısını uygulamanın kendisinin açması gerekmez.

Explicit Wayland Display

Deney ortamı Wayland kullandığı için açık native display bağlantısı şu şekilde oluşturuldu:

```
struct wl_display *wayland_display = wl_display_connect(NULL);

EGLDisplay dpy =
    eglGetDisplay((EGLNativeDisplayType)wayland_display);
```

`wl_display_connect` bir EGL fonksiyonu değildir; Wayland API'sine aittir.

Deneyler

Test 1 — `EGL_DEFAULT_DISPLAY`

```
EGLDisplay defaultDisplay =
    eglGetDisplay(EGL_DEFAULT_DISPLAY);
```

Gerçek çıktı:

```
display_id      : EGL_DEFAULT_DISPLAY
Returned value  : 0x629b25a1cdf0
Result          : SUCCESS
EGL error       : EGL_SUCCESS (0x3000)
```

Test 2 — Aynı `EGL_DEFAULT_DISPLAY` tekrar

```
EGLDisplay defaultDisplayAgain =
    eglGetDisplay(EGL_DEFAULT_DISPLAY);
```

Gerçek çıktı:

```
First handle      : 0x629b25a1cdf0
Second handle     : 0x629b25a1cdf0
Result            : SAME HANDLE
```

Bu, test edilen Mesa implementasyonunda aynı default display isteğinin aynı handle ile sonuçlandığını gösterir.

Bu davranışın bütün EGL implementasyonlarında aynı şekilde gerçekleşeceği varsayılmamalıdır.

Test 3 — Explicit Wayland Display A

```
struct wl_display *waylandDisplayA =
    wl_display_connect(NULL);

EGLDisplay eglDisplayA =
    eglGetDisplay((EGLNativeDisplayType)waylandDisplayA);
```

Gerçek çıktı:

```
Wayland display A: 0x629b25a1da80

Returned value    : 0x629b25a25de0
Result            : SUCCESS
EGL error         : EGL_SUCCESS (0x3000)
```

Test 4 — Explicit Wayland Display B

```
struct wl_display *waylandDisplayB =
    wl_display_connect(NULL);

EGLDisplay eglDisplayB =
    eglGetDisplay((EGLNativeDisplayType)waylandDisplayB);
```

Gerçek çıktı:

```
Wayland display B: 0x629b25a21c30
EGLDisplay B      : 0x629b25a268b0
```

Karşılaştırma:

```

Default EGLDisplay: 0x629b25a1cdf0
EGLDisplay A      : 0x629b25a25de0
EGLDisplay B      : 0x629b25a268b0

A vs B            : DIFFERENT HANDLES
Default vs A      : DIFFERENT HANDLES
Default vs B      : DIFFERENT HANDLES

```

Deney Sonuçları

Test	display_id	Sonuç
EGL_DEFAULT_DISPLAY	Default native display	Başarılı, geçerli handle
EGL_DEFAULT_DISPLAY tekrar	Aynı input	Başarılı, aynı handle
Wayland Display A	Explicit native display	Başarılı, farklı handle
Wayland Display B	İkinci explicit native display	Başarılı, farklı handle

eglGetDisplay ve eglInitialize İlişkisi

```

eglGetDisplay
|
v
EGLDisplay handle elde edilir
|
v
eglInitialize
|
v
Display EGL kullanımı için initialize edilir

```

`eglGetDisplay` başarılı olsa bile display henüz initialize edilmiş sayılmaz.

Minimal Kullanım

```
EGLDisplay dpy = eglGetDisplay(EGL_DEFAULT_DISPLAY);

if (dpy == EGL_NO_DISPLAY) {
    EGLint err = eglGetError();
}
```

EGL 1.0 İçin Pratik Özet

- `eglGetDisplay` bir native display'den `EGLDisplay` handle'ı elde eder.
- Tek parametresi `display_id` 'dir.
- `display_id` , `EGLNativeDisplayType` türündedir ve platforma bağımlıdır.
- `EGL_DEFAULT_DISPLAY` , varsayılan native display'i istemek için kullanılır.
- Geçerli explicit native display değerleri de verilebilir.
- Başarısızlıkta `EGL_NO_DISPLAY` döner.
- `EGLDisplay` opaque handle'dır; sayısal değeri yorumlanmamalıdır.
- `eglGetDisplay` display'i initialize etmez; sonraki adım tipik olarak `eglInitialize` 'dır.

EGL 1.0: `eglInitialize`

```
EGLBoolean eglInitialize(EGLDisplay dpy,
                        EGLint *major,
                        EGLint *minor);
```

`eglInitialize` , bir `EGLDisplay` bağlantısını EGL kullanımı için initialize eder ve istenirse EGL implementation sürümünü `major` ve `minor` output parametreleri üzerinden döndürür.

Kısa özet:

- `dpy` : initialize edilecek EGL display.
- `major` : major sürüm output pointer'ı.
- `minor` : minor sürüm output pointer'ı.
- Başarılı çağrı: `EGL_TRUE`
- Başarısız çağrı: `EGL_FALSE`
- Geçersiz display deneyinde: `EGL_BAD_DISPLAY`

Mental Model

```
Native display
|
v
eglGetDisplay
|
v
EGLDisplay
|
v
eglInitialize
|
+-- EGL bağlantısı initialize edilir
|
+-- major/minor istenirse sürüm bilgisi alınır
```

Parametreler

dpy

dpy , initialize edilecek geçerli bir EGLDisplay olmalıdır.

dpy	Sonuç
Geçerli fresh EGLDisplay	Initialization yapılabilir.
EGL_NO_DISPLAY	Deneyde EGL_FALSE , EGL_BAD_DISPLAY .

Normal akış:

```
EGLDisplay dpy = eglGetDisplay(EGL_DEFAULT_DISPLAY);
eglInitialize(dpy, &major, &minor);
```

eglGetDisplay için ilgili bölüme bakınız.

major

major , major EGL sürüm numarasının yazılabileceği EGLint * output parametresidir.

```
EGLint major = -1;
eglInitialize(dpy, &major, &minor);
```

Deneylerde -1 yalnızca değişkenin çağrı sırasında değiştirilip değiştirilmediğini görmek için sentinel değer olarak kullanıldı.

minor

`minor` , minor EGL sürüm numarasının yazılabileceği `EGLint *` output parametresidir.

```
EGLint minor = -1;
eglInitialize(dpy, &major, &minor);
```

Normal testte:

```
major = 1
minor = 5
```

elde edildi.

Bu değer test edilen Mesa EGL 1.5 runtime'ına aittir.

NULL Kullanımı

Deneylerde şu kombinasyonlar test edildi:

```
eglInitialize(dpy, &major, &minor);
eglInitialize(dpy, NULL, &minor);
eglInitialize(dpy, &major, NULL);
eglInitialize(dpy, NULL, NULL);
```

Her test için fresh bir Wayland native display bağlantısı oluşturuldu. Böylece önceki initialization çağrılarının sonraki deneylere etkisi azaltıldı.

Deneyler

Test 1 — `major` ve `minor` birlikte

```
EGLint major = -1;
EGLint minor = -1;

EGLBoolean result =
    eglInitialize(display, &major, &minor);
```

Gerçek çıktı:

```
major before      : -1
minor before      : -1
Return value      : EGL_TRUE
major after       : 1
minor after       : 5
EGL error         : EGL_SUCCESS (0x3000)
```

Test 2 — major = NULL

```
EGLint minor = -1;

EGLBoolean result =
    eglInitialize(display, NULL, &minor);
```

Gerçek çıktı:

```
major             : NULL
minor             : &minor
Return value      : EGL_TRUE
minor after       : -1
EGL error         : EGL_SUCCESS (0x3000)
```

Test edilen Mesa implementasyonunda çağrı başarılı oldu ancak `minor` değeri değişmedi.

Test 3 — minor = NULL

```
EGLint major = -1;

EGLBoolean result =
    eglInitialize(display, &major, NULL);
```

Gerçek çıktı:

```
major             : &major
minor             : NULL
Return value      : EGL_TRUE
major after       : -1
EGL error         : EGL_SUCCESS (0x3000)
```

Test edilen Mesa implementasyonunda çağrı başarılı oldu ancak `major` değeri değişmedi.

Bu iki sonuç implementation-specific deney gözlemidir. “Bütün EGL implementasyonları tek output pointer verildiğinde aynı davranışı gösterir” şeklinde genellenmemelidir.

Test 4 — `major = NULL , minor = NULL`

```
EGLBoolean result =
    eglInitialize(display, NULL, NULL);
```

Gerçek çıktı:

```
major           : NULL
minor           : NULL
Return value    : EGL_TRUE
EGL error       : EGL_SUCCESS (0x3000)
```

Bu test, sürüm output'ları alınmadan da initialization yapılabildiğini gösterdi.

Test 5 — `dpy = EGL_NO_DISPLAY`

```
EGLint major = -1;
EGLint minor = -1;

EGLBoolean result =
    eglInitialize(EGL_NO_DISPLAY, &major, &minor);
```

Gerçek çıktı:

```
dpy             : EGL_NO_DISPLAY
Return value     : EGL_FALSE
major after      : -1
minor after      : -1
EGL error        : EGL_BAD_DISPLAY (0x3008)
```

Deney Sonuçları

dpy	major	minor	Sonuç
Geçerli	pointer	pointer	EGL_TRUE , 1.5
Geçerli	NULL	pointer	EGL_TRUE , testte minor değişmedi

dpy	major	minor	Sonuç
Geçerli	pointer	NULL	EGL_TRUE , testte major değişmedi
Geçerli	NULL	NULL	EGL_TRUE
EGL_NO_DISPLAY	pointer	pointer	EGL_FALSE , EGL_BAD_DISPLAY

Hata Matrisi

Durum	Sonuç
Geçerli display, iki output pointer	EGL_TRUE ; testte sürüm 1.5
Geçerli display, iki output pointer NULL	EGL_TRUE
dpy == EGL_NO_DISPLAY	EGL_FALSE , EGL_BAD_DISPLAY
Initialization sırasında driver warning görülmesi	Tek başına EGL API hatası anlamına gelmez.

Mesa / Driver Warning Notu

Bazı testlerde:

```
MESA: error: ZINK: failed to choose pdev
libEGL warning: egl: failed to create dri2 screen
libEGL warning: DRI3 error: Could not get DRI3 device
```

mesajları görüldü.

Buna rağmen ilgili EGL çağrılarında:

```
EGL_TRUE
EGL_SUCCESS
```

elde edilebildi.

Bu nedenle driver warning ile EGL API hata sonucu birbirinden ayrılmalıdır.

Minimal Kullanım

```
EGLDisplay dpy = eglGetDisplay(EGL_DEFAULT_DISPLAY);

if (dpy == EGL_NO_DISPLAY) {
    return 1;
}

EGLint major;
EGLint minor;

if (!eglInitialize(dpy, &major, &minor)) {
    EGLint err = eglGetError();
    return 1;
}

printf("EGL version: %d.%d\n", major, minor);
```

Bilerek Hata Üretme Örneği

```
EGLint major = -1;
EGLint minor = -1;

if (!eglInitialize(EGL_NO_DISPLAY, &major, &minor)) {
    EGLint err = eglGetError(); /* Testte EGL_BAD_DISPLAY */
}
```

EGL 1.0 İçin Pratik Özet

- `eglInitialize` , `EGLDisplay` bağlantısını EGL kullanımı için initialize eder.
- `dpy` , geçerli bir `EGLDisplay` olmalıdır.
- `major` ve `minor` , sürüm bilgisini almak için kullanılan output pointer'larıdır.
- Testte normal kullanım `1.5` döndürdü; bu runtime sürümüdür.
- `major` ve `minor` birlikte `NULL` verildiğinde test edilen implementasyonda çağrı başarılı oldu.
- `EGL_NO_DISPLAY` kullanımı `EGL_FALSE` ve `EGL_BAD_DISPLAY` üretti.
- Driver warning mesajları ile EGL API hata sonucu aynı şey değildir.

EGL 1.0: `eglGetConfigs`

```

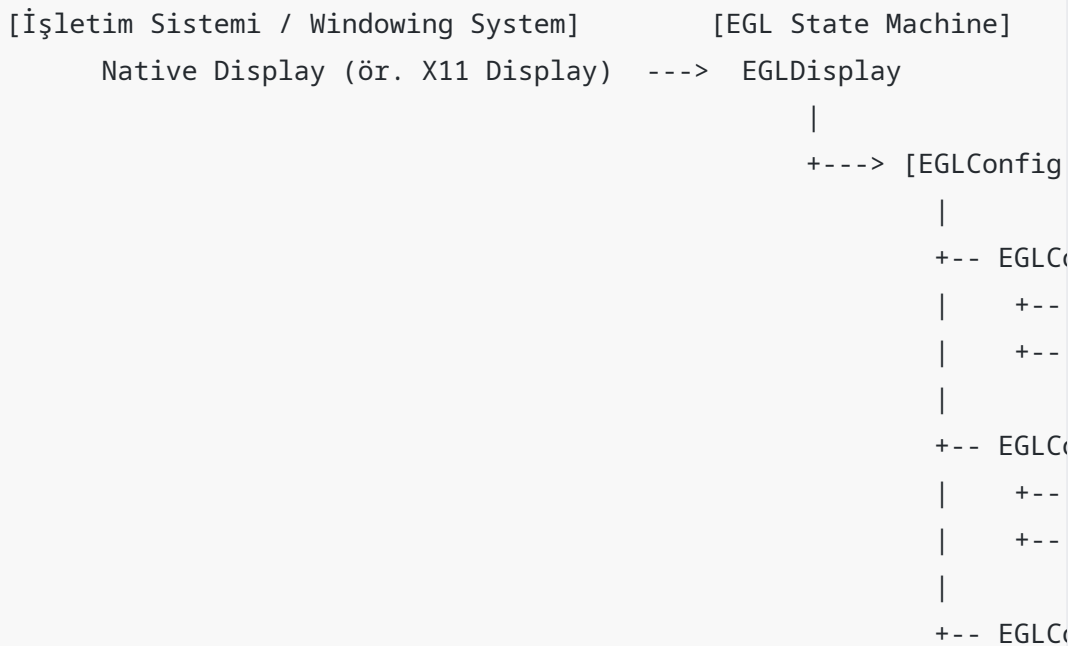
EGLBoolean eglGetConfigs(EGLDisplay dpy,
                        EGLConfig *configs,
                        EGLint config_size,
                        EGLint *num_config);

```

`eglGetConfigs` , daha önceden başlatılmış (initialized) bir EGL görüntüleme bağlantısı (`EGLDisplay`) üzerinden, sistemin donanımsal olarak desteklediği tüm geçerli framebuffer yapılandırmalarının (`EGLConfig`) listesini veya yalnızca mevcut konfigürasyonların toplam sayısını elde etmek için kullanılan temel bir EGL fonksiyonudur.

Mental Model

EGL mimarisinde `eglGetConfigs` fonksiyonu, işletim sisteminin native pencereleme sistemi (X11, Wayland, DRM/GBM veya gömülü pencereleme sistemleri) ile EGL arasındaki uyumlu framebuffer formatlarının bir envanterini sunar.



Bu fonksiyon havuz üzerinde herhangi bir filtreleme veya sıralama yapmaz (bu görev `eglChooseConfig` 'e aittir); yalnızca sistemin native konfigürasyonlarına karşılık gelen tüm `EGLConfig` handle'larını olduğu gibi kullanıcıya sunar.

Parametreler

`dpy` (`EGLDisplay`)

EGL ortamını temsil eden opaque (kapalı) bir handle'dır. İşletim sisteminin native display'i (örneğin X11 `Display*` veya DRM dosya tanımlayıcısı)

kullanılarak `eglGetDisplay` ile elde edilir ve `eglInitialize` ile başlatılmış olmalıdır.

Değer	Sonuç
Geçerli ve Başlatılmış <code>EGLDisplay</code>	İşlem devam eder, diğer argümanlar da doğruysa sorgu başarılıdır.
<code>EGL_NO_DISPLAY</code>	Fonksiyon <code>EGL_FALSE</code> döner. <code>eglGetError()</code> -> <code>EGL_BAD_DISPLAY</code> üretir.
Geçersiz Handle	Fonksiyon <code>EGL_FALSE</code> döner. <code>eglGetError()</code> -> <code>EGL_BAD_DISPLAY</code> üretir.
Başlatılmamış (Uninitialized) Display	Fonksiyon <code>EGL_FALSE</code> döner. <code>eglGetError()</code> -> <code>EGL_NOT_INITIALIZED</code> üretir.

configs (EGLConfig*)

EGL konfigürasyonlarının handle'lerinin (tanımlayıcılarının) yazılacağı bellek bölgesini (dizi) işaret eder.

Değer	Sonuç
Geçerli Bellek Adresi	Sistemdeki mevcut konfigürasyonlar (en fazla <code>config_size</code> kadar) bu diziye kopyalanır.
NULL	Konfigürasyonlar kopyalanmaz. Sadece toplam config sayısı hesaplanıp <code>num_config</code> adresine yazılır (2 adımlı sorgu paterni).

config_size (EGLint)

`configs` dizisine kopyalanabilecek maksimum `EGLConfig` sayısını belirten tam sayıdır.

Değer	Sonuç
0	Eğer <code>configs</code> geçerli bir dizi ise hiçbir veri kopyalanmaz, <code>num_config</code> değeri 0 olur. (Fakat <code>configs</code> NULL ise EGL standardınca dikkate alınmaz).

Değer	Sonuç
Toplam Config < config_size	Sistemdeki tüm config'ler diziye yazılır. num_config içine kopyalanan miktar yazılır. Kalan dizi kapasitesi değiştirilmez.
Toplam Config > config_size	EGL yalnızca config_size kadar config'i diziye kopyalar. Taşanlar göz ardı edilir, herhangi bir hata üretilmez. num_config değeri config_size değerine eşit olur.

num_config (EGLint*)

EGL'nin "Bu diziye kaç konfigürasyon yazdım?" (eğer configs != NULL ise) veya "Sistemde toplam kaç konfigürasyon var?" (eğer configs == NULL ise) sorusuna cevabını ilettiği çıkış (output) parametresidir.

Değer	Sonuç
Geçerli EGLint* adresi	Elde edilen sayı bu bellek adresine başarıyla yazılır.
NULL	Khronos standardına göre kesinlikle yasaktır. Çökme (segmentation fault) veya EGL_BAD_PARAMETER üretir.

Geçerli Attribute Listesi ve Tampon (Buffer) Tipleri

eglGetConfigs fonksiyonu herhangi bir nitelik (attribute) listesi argümanı almaz. Sistemin desteklediği tüm EGL konfigürasyonlarını filtresiz döndürür. Elde edilen her bir EGLConfig , eglGetConfigAttrib kullanılarak tek tek incelenebilir.

Elde edilecek konfigürasyonlar donanımın desteklediği aşağıdaki tampon tiplerine sahip olabilir ve aralarındaki davranış farklılıkları kritik öneme sahiptir:

- **Back Buffered (Çift Tamponlu):** Pencere yüzeyleri (Window Surfaces) için uygundur. Ekranda yırtılma (tearing) olmaması için eglSwapBuffers işlemi gerektirir.
- **Single Buffered (Tek Tamponlu):** Pixmap yüzeyleri (Pixmap Surfaces) için uygundur. Çizilen her şey anında native belleğe yansır.

Hangisinin hangi tampon tipini desteklediğini anlamak için dönen config'lerin EGL_SURFACE_TYPE niteliği (EGL_WINDOW_BIT , EGL_PIXMAP_BIT , EGL_PBUFFER_BIT) kontrol edilmelidir.

Ayrıntılar ve Yaşam Döngüsü

Thread Güvenliği (Thread Safety): EGL'de konfigürasyon elde etme işlemleri (örneğin `eglGetConfigs`) thread-safe'tir. Birden fazla thread aynı `EGLDisplay` üzerinden eşzamanlı (concurrent) olarak bu fonksiyonu çağırabilir. Veri EGL state machine'ini modifiye etmeyip sadece okunup döndürüldüğü için bir yarış koşulu yaratmaz.

Eşzamanlama (Synchronization): Bu fonksiyon donanıma veya Native render motoruna komut (command buffer) göndermez, sürücünün (driver) önceden oluşturduğu statik konfigürasyon listesini okur. Bu yüzden `glFinish()` , `eglWaitGL()` veya `eglWaitNative()` gibi komutların çağırılmasına gerek yoktur.

Yaşam Döngüsü: Döndürülen `EGLConfig` handle'ları, o handle'ları barındıran `EGLDisplay` var olduğu sürece geçerlidir. Display `eglTerminate` ile sonlandırıldığında bu config'ler geçersizleşir ve bir daha kullanılamazlar.

Hata Matrisi

EGL spesifikasyonunun katı kuralları gereği `eglGetConfigs` başarısız olduğunda **hiçbir yan etki (side effect) bırakmamalıdır**. Argümanlardaki bellek alanları değiştirilmez ve state machine mevcut durumunu aynen korur.

Durum	Sonuç (Fonksiyon Dönüşü)	EGL Hata Kodu (<code>eglGetError()</code>)
Her şey geçerli ve başarılı	<code>EGL_TRUE</code>	<code>EGL_SUCCESS</code>
<code>dpy</code> geçerli bir display değil veya <code>EGL_NO_DISPLAY</code>	<code>EGL_FALSE</code>	<code>EGL_BAD_DISPLAY</code>
<code>dpy</code> henüz <code>eglInitialize</code> ile başlatılmamış	<code>EGL_FALSE</code>	<code>EGL_NOT_INITIALIZED</code>
<code>num_config</code> parametresinin <code>NULL</code> olması	<code>EGL_FALSE</code> (veya Crash)	<code>EGL_BAD_PARAMETER</code>

Güvenli Kullanım Örneği

Aşağıdaki C kodu, EGL spesifikasyonuna tam uyumlu, hafıza sızıntısı (memory leak) riskini sıfırlayan, modern EGL yazılımlarındaki **İki Adımlı Sorgu (Two-Step**

Query) yaklaşımını gösterir.

```
#include <EGL/egl.h>
#include <stdio.h>
#include <stdlib.h>

/*
 * Sistemdeki tüm konfigürasyonları elde edip ekrana sayısını yazdır.
 * Başarılı olursa EGL_TRUE, aksi halde EGL_FALSE döner.
 */
EGLBoolean SafeGetEGLConfigs(EGLDisplay dpy) {
    EGLint total_configs = 0;
    EGLConfig *configs = NULL;

    // 1. ADIM: Sadece toplam config sayısını sor (configs = NULL)
    if (eglGetConfigs(dpy, NULL, 0, &total_configs) != EGL_TRUE) {
        fprintf(stderr, "[Hata] EGL config sayısı okunamadı. Hata: %d\n",
            eglGetError());
        return EGL_FALSE;
    }

    if (total_configs == 0) {
        printf("[Bilgi] EGL başlatılmış ancak desteklenen konfigürasyon yok.\n");
        return EGL_TRUE; // Bu bir donanım limitidir, API hatası değildir.
    }

    // 2. ADIM: Dönen sayı kadar heap'ten dinamik bellek tahsis et
    configs = (EGLConfig*) malloc(total_configs * sizeof(EGLConfig));
    if (configs == NULL) {
        fprintf(stderr, "[Kritik Hata] Bellek yetersizliği (Out of Memory).\n");
        return EGL_FALSE;
    }

    // 3. ADIM: Ayrılan belleği gerçek konfigürasyonlarla doldur
    if (eglGetConfigs(dpy, configs, total_configs, &total_configs) != EGL_TRUE) {
        fprintf(stderr, "[Hata] EGL config verileri okunamadı. Hata: %d\n",
            eglGetError());
        free(configs);
        return EGL_FALSE;
    }

    printf("Başarılı: Sistemden %d adet EGLConfig çekildi.\n", total_configs);

    // TODO: Burada 'configs' dizisi eglGetConfigAttrib ile filtrelenmeli.
    // projenin ihtiyaç duyduğu Back Buffered veya pBuffer özelliği için.

    // İşlem bittikten sonra ayrılan belleği daima temizle (Memory
```

```
free(configs);

return EGL_TRUE;
}
```

Pratik Özet

- **Amaç ve Sınırlar:** `eglGetConfigs` , veritabanındaki tüm konfigürasyonları hiçbir filtre uygulamadan ham (raw) formatta listeler. Özel ihtiyaçlar (ör. Z-Buffer'ı 24 bit olan configler) aranıyorsa her zaman `eglChooseConfig` tercih edilmelidir.
- **İki Adımlı Sorgu (Best Practice):** Gömülü ve safety-critical sistemlerde rastgele boyutlu statik diziler (ör. `EGLConfig arr[100]`) gereksiz bellek israfına veya array taşmasına yol açabilir. Her zaman önce `configs = NULL` ile sayıyı öğrenin ve dinamik olarak yeterli alan ayırın.
- **Null Pointer Kısıtı:** `num_config` argümanı hiçbir senaryoda (sadece sayım yaparken bile) `NULL` olamaz, adres geçmek zorunludur. Aksi takdirde uygulamanız EGL standardını ihlal eder ve crash olabilir.
- **Yan Etkisizlik:** Fonksiyon salt-okunur (read-only) tabanlı bir işlemdir. Başarısız çağrılar hiçbir state değişikliğine veya mevcut bellek alanında bozulmaya neden olmaz.

EGL 1.0: `eglChooseConfig`

```
EGLBoolean eglChooseConfig(EGLDisplay dpy,
                           const EGLint *attrib_list,
                           EGLConfig *configs,
                           EGLint config_size,
                           EGLint *num_config);
```

`eglChooseConfig` , bir `EGLDisplay` üzerinde bulunan EGL configuration'ları uygulamanın verdiği attribute kriterlerine göre filtreleyip sıralamak için kullanılır.

Kısa özet:

- `dpy` : config'lerin aranacağı initialized display.
- `attrib_list` : attribute/value çiftleri.
- `configs` : eşleşen `EGLConfig` handle'larının yazılacağı buffer.
- `config_size` : output buffer kapasitesi.

- `num_config` : döndürülen config sayısının yazıldığı output pointer.
- Uygun config bulunmaması API hatası olmak zorunda değildir.
- `EGL_TRUE + num_config = 0` geçerli bir sonuçtur.

Mental Model

```

EGLDisplay
|
+-- Config #1
+-- Config #2
+-- Config #3
+-- ...
    |
    v
    attrib_list kriterleri
    |
    v
    eglChooseConfig
    |
    v
    Eşleşen EGLConfig'ler
  
```

Parametreler

`dpy`

`dpy` , config seçim işleminin yapılacağı initialized `EGLDisplay` 'dir.

Değer	Sonuç
Geçerli initialized display	Config seçimi yapılabilir.
<code>EGL_NO_DISPLAY</code>	Deneyde <code>EGL_FALSE</code> , <code>EGL_BAD_DISPLAY</code> .

`attrib_list`

`attrib_list` , attribute/value çiftlerinden oluşur ve `EGL_NONE` ile sonlandırılır.

Örnek:

```
const EGLint attrs[] = {
    EGL_RED_SIZE, 8,
    EGL_GREEN_SIZE, 8,
    EGL_BLUE_SIZE, 8,
    EGL_NONE
};
```

Kavramsal görünüm:

Attribute	Value

EGL_RED_SIZE	8
EGL_GREEN_SIZE	8
EGL_BLUE_SIZE	8
EGL_NONE	-

EGL_NONE

Listenin sonunu belirtir:

```
const EGLint attrs[] = {
    EGL_RED_SIZE, 8,
    EGL_NONE
};
```

attrib_list = NULL

Deney:

```
eglChooseConfig(dpy, NULL, NULL, 0, &num_config);
```

Gerçek sonuç:

```
Return value      : EGL_TRUE
Matching configs  : 50
EGL error         : EGL_SUCCESS (0x3000)
```

Belirtilmeyen attribute'lar EGL'nin default seçim değerleriyle değerlendirilir.

configs

configs , eşleşen EGLConfig handle'larının yazılacağı output buffer'dır.

```
EGLConfig configs[5];
```

Yalnızca toplam eşleşme sayısını öğrenmek için `NULL` kullanılabilir:

```
eglChooseConfig(dpy, attrs, NULL, 0, &count);
```

`config_size`

`config_size`, `configs` buffer'ının kapasitesidir.

```
EGLConfig configs[5];
eglChooseConfig(dpy, attrs, configs, 5, &num_config);
```

`config_size = 5`, "5 config bul" değil, "output buffer en fazla 5 config alabilir" anlamına gelir.

`num_config`

`num_config`, döndürülen config sayısının yazıldığı `EGLint *` output parametresidir.

```
EGLint num_config = -1;
eglChooseConfig(dpy, attrs, configs, 5, &num_config);
```

Deneyde `num_config = NULL` :

```
EGL_FALSE
EGL_BAD_PARAMETER
```

sonucunu verdi.

EGL 1.0 Seçim Attribute'ları

Bu çalışmada özellikle şu EGL 1.0 attribute'ları test edildi:

Attribute	Anlam
<code>EGL_RED_SIZE</code>	Red component bit sayısı için minimum kriter
<code>EGL_GREEN_SIZE</code>	Green component bit sayısı için minimum kriter
<code>EGL_BLUE_SIZE</code>	Blue component bit sayısı için minimum kriter
<code>EGL_ALPHA_SIZE</code>	Alpha component bit sayısı için minimum kriter

Attribute	Anlam
EGL_DEPTH_SIZE	Depth buffer bit sayısı için minimum kriter
EGL_STENCIL_SIZE	Stencil buffer bit sayısı için minimum kriter
EGL_LEVEL	Framebuffer level
EGL_NONE	Attribute listesinin sonu
EGL_DONT_CARE	Uygun attribute'larda bu kriteri önemseme

Seçilen config'in gerçek attribute değerlerini okumak için `eglGetConfigAttrib` kullanılır. Ayrıntılar için `eglGetConfigAttrib` bölümüne bakınız.

Size Attribute'larında Minimum Mantığı

Şu ifade:

```
EGL_RED_SIZE, 8
```

“red değeri kesin olarak 8 olmalı” şeklinde yorumlanmamalıdır.

Size attribute'larında istek minimum gereksinim olarak değerlendirilir.

Bu nedenle:

```
EGL_RED_SIZE >= 8
```

şeklinde düşünmek daha doğrudur.

`attrib_list` Deneyleri

Aynı çağrı yapısı kullanıldı:

```
eglChooseConfig(  
    display,  
    attributes,  
    NULL,  
    0,  
    &num_config  
);
```

Böylece `configs` ve `config_size` sabit tutularak yalnızca `attrib_list`

etkisi incelendi.

Test 1 — `attrib_list = NULL`

```
Matching configs : 50
Return value      : EGL_TRUE
EGL error         : EGL_SUCCESS
```

Test 2 — RGB 8/8/8

```
const EGLint rgb888[] = {
    EGL_RED_SIZE,    8,
    EGL_GREEN_SIZE,  8,
    EGL_BLUE_SIZE,   8,
    EGL_NONE
};
```

Sonuç:

```
Matching configs : 50
```

Test 3 — RGB 5/6/5

```
const EGLint rgb565[] = {
    EGL_RED_SIZE,    5,
    EGL_GREEN_SIZE,  6,
    EGL_BLUE_SIZE,   5,
    EGL_NONE
};
```

Sonuç:

```
Matching configs : 50
```

Test 4 — RGBA 8/8/8/8

```
const EGLint rgba8888[] = {
    EGL_RED_SIZE,    8,
    EGL_GREEN_SIZE,  8,
    EGL_BLUE_SIZE,   8,
    EGL_ALPHA_SIZE,  8,
    EGL_NONE
};
```

Sonuç:

Matching configs : 30

Test 5 — RGB888 + Depth 16

```
const EGLint depth16[] = {
    EGL_RED_SIZE,    8,
    EGL_GREEN_SIZE,  8,
    EGL_BLUE_SIZE,   8,
    EGL_DEPTH_SIZE,  16,
    EGL_NONE
};
```

Sonuç:

Matching configs : 40

Test 6 — RGB888 + Depth 24

```
const EGLint depth24[] = {
    EGL_RED_SIZE,    8,
    EGL_GREEN_SIZE,  8,
    EGL_BLUE_SIZE,   8,
    EGL_DEPTH_SIZE,  24,
    EGL_NONE
};
```

Sonuç:

Matching configs : 30

Test 7 — RGB888 + Depth 24 + Stencil 8

```
const EGLint depth24_stencil8[] = {
    EGL_RED_SIZE,      8,
    EGL_GREEN_SIZE,    8,
    EGL_BLUE_SIZE,     8,
    EGL_DEPTH_SIZE,    24,
    EGL_STENCIL_SIZE,  8,
    EGL_NONE
};
```

Sonuç:

Matching configs : 10

Test 8 — Aşırı büyük gereksinimler

```
const EGLint impossible_config[] = {
    EGL_RED_SIZE,      64,
    EGL_GREEN_SIZE,    64,
    EGL_BLUE_SIZE,     64,
    EGL_DEPTH_SIZE,    64,
    EGL_STENCIL_SIZE,  32,
    EGL_NONE
};
```

Sonuç:

Return value : EGL_TRUE
Matching configs : 0
EGL error : EGL_SUCCESS (0x3000)

Bu test önemli bir ayrımı gösterir:

0 config bulundu
!=
eglChooseConfig başarısız oldu

Test 9 — Alpha `EGL_DONT_CARE`

```
const EGLint dont_care_alpha[] = {
    EGL_RED_SIZE,    8,
    EGL_GREEN_SIZE,  8,
    EGL_BLUE_SIZE,   8,
    EGL_ALPHA_SIZE,  EGL_DONT_CARE,
    EGL_NONE
};
```

Sonuç:

Matching configs : 50

Karşılaştırma:

```
Alpha >= 8      -> 30
Alpha DONT_CARE -> 50
```

`attrib_list` Sonuç Tablosu

Kriter	Eşleşen config
<code>attrib_list = NULL</code>	50
RGB 8/8/8	50
RGB 5/6/5	50
RGBA 8/8/8/8	30
RGB888 + Depth 16	40
RGB888 + Depth 24	30
RGB888 + Depth 24 + Stencil 8	10
Çok yüksek gereksinimler	0
RGB888 + Alpha <code>EGL_DONT_CARE</code>	50

Bu sistemde kriterler sıkılaştıkça genel olarak eşleşen config sayısı azaldı.

Bu sayıların implementasyona bağlı olduğu unutulmamalıdır.

`configs` , `config_size` , `num_config` Deneyleri

Bu testlerde attribute listesi sabit tutuldu:

```
const EGLint attributes[] = {  
    EGL_RED_SIZE,      8,  
    EGL_GREEN_SIZE,    8,  
    EGL_BLUE_SIZE,     8,  
    EGL_DEPTH_SIZE,    24,  
    EGL_STENCIL_SIZE,  8,  
    EGL_NONE  
};
```

Bu kriterlerle toplam 10 config eşleşti.

```
configs = NULL , config_size = 0
```

```
eglChooseConfig(  
    display,  
    attributes,  
    NULL,  
    0,  
    &num1  
);
```

Sonuç:

```
num_config after : 10  
Return value      : EGL_TRUE  
EGL error         : EGL_SUCCESS
```

Bu kullanım toplam eşleşme sayısını sorgular.

```
config_size = 1
```

```
EGLConfig configs[1];  
eglChooseConfig(display, attributes, configs, 1, &num_config);
```

Sonuç:

```
num_config after : 1  
configs[0]       : 0x55df27b2fa30
```

```
config_size = 5
```

Sonuç:

```
num_config after : 5

configs[0] : 0x55df27b2fa30
configs[1] : 0x55df27b2fd70
configs[2] : 0x55df27b2ff50
configs[3] : 0x55df27b31b30
configs[4] : 0x55df27b2f210
```

config_size = 100

Toplam uygun config sayısı 10 olduğu için:

```
num_config after : 10
Return value      : EGL_TRUE
EGL error         : EGL_SUCCESS
```

elde edildi.

config_size Karşılaştırması

Toplam uygun config	config_size	num_config
10	1	1
10	5	5
10	100	10

num_config = NULL

```
eglChooseConfig(
    display,
    attributes,
    configs,
    5,
    NULL
);
```

Gerçek sonuç:

```
Return value : EGL_FALSE
EGL error    : EGL_BAD_PARAMETER (0x300C)
```

Negatif `config_size`

Deney:

```
eglChooseConfig(
    display,
    attributes,
    configs,
    -1,
    &num_config
);
```

Gerçek sonuç:

```
config_size      : -1
num_config after : -1
Return value     : EGL_TRUE
EGL error        : EGL_SUCCESS
```

Bu davranış test edilen Mesa implementasyonunda gözlemlendi.

Negatif buffer kapasitesi normal ve portable bir kullanım değildir; bu sonucu EGL uygulamalarında güvenilecek bir davranış olarak kullanmamak gerekir.

Hata Deneyleri

`dpy = EGL_NO_DISPLAY`

```
eglChooseConfig(
    EGL_NO_DISPLAY,
    valid_attributes,
    NULL,
    0,
    &num_config
);
```

Sonuç:

```
Return value : EGL_FALSE
num_config   : -1
EGL error    : EGL_BAD_DISPLAY (0x3008)
```

Tanınmayan attribute

```
const EGLint unknown_attribute[] = {
    0x12345678, 1,
    EGL_NONE
};
```

Sonuç:

```
Return value : EGL_FALSE
num_config   : -1
EGL error    : EGL_BAD_ATTRIBUTE (0x3004)
```

EGL_LEVEL = EGL_DONT_CARE

```
const EGLint invalid_level[] = {
    EGL_LEVEL, EGL_DONT_CARE,
    EGL_NONE
};
```

Sonuç:

```
Return value : EGL_FALSE
num_config   : -1
EGL error    : EGL_BAD_ATTRIBUTE (0x3004)
```

Hata Sonrası Kontrol

Hata testlerinden sonra normal RGB888 seçimi tekrar yapıldı:

```
Return value : EGL_TRUE
num_config   : 50
EGL error    : EGL_SUCCESS
```

Hata Matrisi

Durum	Sonuç
Geçerli display + geçerli attribute listesi	EGL_TRUE
Uygun config yok	EGL_TRUE , num_config = 0
dpy == EGL_NO_DISPLAY	EGL_FALSE , EGL_BAD_DISPLAY
Tanınmayan attribute	EGL_FALSE , EGL_BAD_ATTRIBUTE
Geçersiz attribute/value	EGL_FALSE , EGL_BAD_ATTRIBUTE
num_config == NULL	EGL_FALSE , EGL_BAD_PARAMETER
Negatif config_size	Mesa testinde EGL_TRUE , output değişmedi; portable kullanım değildir

Minimal Sayı Sorgulama

```
EGLint count = 0;

if (!eglChooseConfig(
    dpy,
    attrs,
    NULL,
    0,
    &count)) {
    EGLint err = eglGetError();
}
```

Minimal Config Alma

```
EGLConfig configs[5];
EGLint count = 0;

if (!eglChooseConfig(
    dpy,
    attrs,
    configs,
    5,
    &count)) {
    EGLint err = eglGetError();
}
```

`eglGetConfigAttrib` ile İlişki

`eglChooseConfig` , config seçer.

Seçilen config'in gerçek özelliklerini okumak için:

```
eglGetConfigAttrib(dpy, config, EGL_RED_SIZE, &value);
```

kullanılır.

`eglGetConfigAttrib` fonksiyonunun ayrıntıları için ilgili bölüme bakınız.

EGL 1.0 İçin Pratik Özet

- `eglChooseConfig` beş parametre alır.
- `attrib_list` , attribute/value çiftlerinden oluşur ve `EGL_NONE` ile biter.
- Size attribute'ları minimum gereksinim gibi değerlendirilir.
- `configs = NULL` , yalnızca eşleşme sayısını sorgulamak için kullanılabilir.
- `config_size` , output buffer kapasitesidir.
- `num_config` , döndürülen config sayısını verir.
- `EGL_TRUE` dönmesi mutlaka config bulunduğu anlamına gelmez.
- `EGL_TRUE + num_config = 0` geçerli bir sonuçtur.
- `num_config = NULL` deneyde `EGL_BAD_PARAMETER` üretti.
- Geçersiz display `EGL_BAD_DISPLAY` üretti.
- Geçersiz attribute veya attribute/value kombinasyonu `EGL_BAD_ATTRIBUTE` üretti.
- Config sayıları ve handle değerleri implementasyona bağlıdır.

EGL 1.0: `eglGetConfigAttrib`

```
EGLBoolean eglGetConfigAttrib(EGLDisplay dpy,  
                             EGLConfig config,  
                             EGLint attribute,  
                             EGLint *value);
```

`eglGetConfigAttrib` , bir `EGLConfig` içindeki tek bir attribute değerini sorgular.

`EGLConfig` , oluşturulacak `EGLSurface` için şu özellikleri tarif eder:

- color buffer component boyutları
- depth/stencil buffer boyutları
- multisampling bilgisi
- desteklenen surface tipleri
- native visual bilgileri
- transparency bilgileri
- pBuffer limitleri

Mental Model

```
EGLDisplay  
|  
+-- EGLConfig #1  
|   +-- EGL_RED_SIZE  
|   +-- EGL_GREEN_SIZE  
|   +-- EGL_SURFACE_TYPE  
|   +-- ...  
|  
+-- EGLConfig #2  
    +-- EGL_RED_SIZE  
    +-- EGL_GREEN_SIZE  
    +-- EGL_SURFACE_TYPE  
    +-- ...
```

`eglGetConfigAttrib` sadece bir `EGLConfig` ve bir `attribute` için değer döndürür.

Parametreler

dpy

Değer	Sonuç
Geçerli ve initialized <code>EGLDisplay</code>	Diğer parametreler de geçerliyse sorgu başarılıdır.
<code>EGL_NO_DISPLAY</code>	Başarısız. Genel EGL hata modeliyle <code>EGL_BAD_DISPLAY</code> beklenir.
Geçersiz display handle	Başarısız. Genel EGL hata modeliyle <code>EGL_BAD_DISPLAY</code> beklenir.
Initialize edilmemiş display	Başarısız. Genel EGL hata modeliyle <code>EGL_NOT_INITIALIZED</code> beklenir.

config

Değer	Sonuç
<code>dpy</code> üzerinden alınmış geçerli <code>EGLConfig</code>	Attribute değeri <code>*value</code> içine yazılır.
<code>eglGetConfigs</code> ile alınmış config	Geçerli.
<code>eglChooseConfig</code> ile alınmış config	Geçerli.
Başka <code>EGLDisplay</code> 'e ait config	Geçerli config sayılmaz; başarısız.
Geçersiz config handle	Başarısız. Tipik hata <code>EGL_BAD_CONFIG</code> .

EGL 1.0 notu:

`EGLConfig` handle'ları, ait oldukları `EGLDisplay` terminate edilene l

attribute

`attribute` , EGL 1.0 Table 3.1 içinde tanımlı bir `EGLConfig` attribute'u olmalıdır. Geçersizse:

```
eglGetConfigAttrib(...) == EGL_FALSE
eglGetError() == EGL_BAD_ATTRIBUTE
```

value

Değer	Sonuç
Geçerli EGLint *	Sonuç bu adrese yazılır.
NULL	EGL 1.0 bunu geçerli kullanım olarak tanımlamaz; gerçek storage ver.

Doğru kullanım:

```
EGLint red_bits = 0;
if (!eglGetConfigAttrib(dpy, config, EGL_RED_SIZE, &red_bits)) {
    EGLint err = eglGetError();
}
```

EGL 1.0 Geçerli Attribute Listesi

Attribute	Tip	Anlam
EGL_BUFFER_SIZE	integer	Color buffer toplam bit derinliği.
EGL_RED_SIZE	integer	Red component bit sayısı.
EGL_GREEN_SIZE	integer	Green component bit sayısı.
EGL_BLUE_SIZE	integer	Blue component bit sayısı.
EGL_ALPHA_SIZE	integer	Alpha component bit sayısı.
EGL_CONFIG_CAVEAT	enum	EGL_NONE , EGL_SLOW_CONFIG , EGL_NON_CONFORMANT_CONFIG .
EGL_CONFIG_ID	integer	Unique config id.
EGL_DEPTH_SIZE	integer	Depth buffer bit sayısı.
EGL_LEVEL	integer	Framebuffer level.
EGL_MAX_PBUFFER_WIDTH	integer	Maksimum pbuffer genişliği.
EGL_MAX_PBUFFER_HEIGHT	integer	Maksimum pbuffer yüksekliği.

Attribute	Tip	Anlam
EGL_MAX_PBUFFER_PIXELS	integer	Maksimum pbuffer pixel sayısı.
EGL_NATIVE_RENDERABLE	boolean	Native rendering API surface'e render edebilir mi?
EGL_NATIVE_VISUAL_ID	integer	Platform-dependent native visual id.
EGL_NATIVE_VISUAL_TYPE	integer	Platform-dependent native visual type.
EGL_SAMPLE_BUFFERS	integer	Multisample buffer sayısı; 0 veya 1 .
EGL_SAMPLES	integer	Pixel başına sample sayısı.
EGL_STENCIL_SIZE	integer	Stencil buffer bit sayısı.
EGL_SURFACE_TYPE	bitmask	Desteklenen surface tipleri.
EGL_TRANSPARENT_TYPE	enum	EGL_NONE veya EGL_TRANSPARENT_RGB .
EGL_TRANSPARENT_RED_VALUE	integer	Transparent red key.
EGL_TRANSPARENT_GREEN_VALUE	integer	Transparent green key.
EGL_TRANSPARENT_BLUE_VALUE	integer	Transparent blue key.

Attribute Ayrıntıları

Color Buffer

```
EGL_BUFFER_SIZE = EGL_RED_SIZE
                  + EGL_GREEN_SIZE
                  + EGL_BLUE_SIZE
                  + EGL_ALPHA_SIZE
```

Örnek:

```
R=8, G=8, B=8, A=8 -> EGL_BUFFER_SIZE = 32
R=5, G=6, B=5, A=0 -> EGL_BUFFER_SIZE = 16
```

EGL_CONFIG_CAVEAT

Değer	Anlam
EGL_NONE	Config için caveat yok.
EGL_SLOW_CONFIG	Render yavaş olabilir. Örneğin donanım doğrudan desteklemiyor olabilir.
EGL_NON_CONFORMANT_CONFIG	OpenGL ES conformance gereksinimlerini karşılamayabilir.

EGL_SURFACE_TYPE

Bitmask döner:

```
EGLint surface_type = 0;
eglGetConfigAttrib(dpy, config, EGL_SURFACE_TYPE, &surface_type);

if (surface_type & EGL_WINDOW_BIT) {
    /* eglCreateWindowSurface için uygundur */
}

if (surface_type & EGL_PIXMAP_BIT) {
    /* eglCreatePixmapSurface için uygundur */
}

if (surface_type & EGL_PBUFFER_BIT) {
    /* eglCreatePbufferSurface için uygundur */
}
```

Geçerli bitler:

Bit	Anlam
EGL_WINDOW_BIT	Window surface oluşturulabilir.
EGL_PIXMAP_BIT	Pixmap surface oluşturulabilir.
EGL_PBUFFER_BIT	Pbuffer surface oluşturulabilir.

Native Visual

EGL_NATIVE_VISUAL_ID ve EGL_NATIVE_VISUAL_TYPE platforma bağlıdır.

EGL 1.0 davranışı:

Durum	EGL_NATIVE_VISUAL_ID	EGL_NATIVE_VISUAL_TYPE
Config window destekliyor ve native visual varsa	Platform-specific id	Platform-specific type
Config window desteklemiyor	0	EGL_NONE
Associated native visual yok	0	EGL_NONE

GBM kullanırken modern Mesa EGL tarafında `EGL_NATIVE_VISUAL_ID` pratikte GBM/DRM formatını seçmek için kullanışlı olabilir. Bu EGL 1.0 spec'in platform-dependent native visual alanına girer.

Multisampling

Attribute	Anlam
<code>EGL_SAMPLE_BUFFERS</code>	0 ise multisample yok, 1 ise multisample buffer var.
<code>EGL_SAMPLES</code>	Sample sayısı. <code>EGL_SAMPLE_BUFFERS == 0</code> ise <code>EGL_SAMPLES</code> da 0 olur.

Transparency

Attribute	Anlam
<code>EGL_TRANSPARENT_TYPE == EGL_NONE</code>	Transparent pixel desteği yok.
<code>EGL_TRANSPARENT_TYPE == EGL_TRANSPARENT_RGB</code>	RGB key değerleri transparent pixel üretir.

`EGL_TRANSPARENT_TYPE == EGL_NONE` ise şu değerler tanımsızdır:

- `EGL_TRANSPARENT_RED_VALUE`
- `EGL_TRANSPARENT_GREEN_VALUE`
- `EGL_TRANSPARENT_BLUE_VALUE`

`EGL_TRANSPARENT_TYPE == EGL_TRANSPARENT_RGB` ise bu değerler component bit derinliği aralığında integer framebuffer değerleridir.

EGL 1.0 Attribute Değilse

Şu token'lar modern EGL header'larında olabilir ama EGL 1.0 Table 3.1 config attribute listesinde yoktur:

Token	Neden dikkat edilmeli
EGL_RENDERABLE_TYPE	EGL 1.0 config attribute listesinde yoktur; sonraki EGL sürümlerinde yaygındır.
EGL_BIND_TO_TEXTURE_RGB	EGL 1.0 Table 3.1 parçası değildir.
EGL_BIND_TO_TEXTURE_RGBA	EGL 1.0 Table 3.1 parçası değildir.
EGL_MIN_SWAP_INTERVAL	EGL 1.0 Table 3.1 parçası değildir.
EGL_MAX_SWAP_INTERVAL	EGL 1.0 Table 3.1 parçası değildir.
EGL_CONTEXT_CLIENT_VERSION	Context creation attribute'tur; EGL 1.0 Table 3.1 config attribute'u değildir.
EGL_WIDTH	Surface attribute'tur; config attribute değildir.
EGL_HEIGHT	Surface attribute'tur; config attribute değildir.

EGL 1.0 uyumu hedefleniyorsa `eglGetConfigAttrib` için yukarıdaki geçerli liste dışına çıkma.

`eglChooseConfig` ile İlişki

`eglGetConfigAttrib` , seçilmiş config'i okur. Config seçme işi `eglChooseConfig` veya `eglGetConfigs` ile yapılır.

EGL 1.0 default/match davranışlarından bazıları:

Attribute	Default	Match tipi
EGL_BUFFER_SIZE	0	En az istenen değeri karşılayan config'ler.
EGL_RED_SIZE	0	En az istenen değeri karşılayan config'ler.

Attribute	Default	Match tipi
EGL_GREEN_SIZE	0	En az istenen değeri karşılayan config'ler.
EGL_BLUE_SIZE	0	En az istenen değeri karşılayan config'ler.
EGL_ALPHA_SIZE	0	En az istenen değeri karşılayan config'ler.
EGL_DEPTH_SIZE	0	En az istenen değeri karşılayan config'ler.
EGL_STENCIL_SIZE	0	En az istenen değeri karşılayan config'ler.
EGL_SURFACE_TYPE	EGL_WINDOW_BIT	Mask match.
EGL_TRANSPARENT_TYPE	EGL_NONE	Exact match.

Bu tablo `eglGetConfigAttrib` çağrısının kendisini değil, sorguladığın config'in nasıl seçilmiş olabileceğini anlamayı kolaylaştırır.

Hata Matrisi

Durum	Sonuç
dpy geçerli, config geçerli, attribute geçerli, value geçerli	EGL_TRUE , *value yazılır.
attribute EGL 1.0 config attribute'u değil	EGL_FALSE , EGL_BAD_ATTRIBUTE .
config geçersiz	EGL_FALSE , tipik hata EGL_BAD_CONFIG .
dpy geçersiz	EGL_FALSE , tipik hata EGL_BAD_DISPLAY .
dpy initialize edilmemiş	EGL_FALSE , tipik hata EGL_NOT_INITIALIZED .
value == NULL	EGL 1.0 geçerli kullanım olarak tanımlamaz; kullanma.

Minimal Kullanım

```
EGLint red = 0;
EGLint green = 0;
EGLint blue = 0;
EGLint surface_type = 0;

eglGetConfigAttrib(dpy, config, EGL_RED_SIZE, &red);
eglGetConfigAttrib(dpy, config, EGL_GREEN_SIZE, &green);
eglGetConfigAttrib(dpy, config, EGL_BLUE_SIZE, &blue);
eglGetConfigAttrib(dpy, config, EGL_SURFACE_TYPE, &surface_type);
```

Bütün EGL 1.0 Attribute'larını Sorgulama

```
static const EGLint attrs[] = {
    EGL_BUFFER_SIZE,
    EGL_RED_SIZE,
    EGL_GREEN_SIZE,
    EGL_BLUE_SIZE,
    EGL_ALPHA_SIZE,
    EGL_CONFIG_CAVEAT,
    EGL_CONFIG_ID,
    EGL_DEPTH_SIZE,
    EGL_LEVEL,
    EGL_MAX_PBUFFER_WIDTH,
    EGL_MAX_PBUFFER_HEIGHT,
    EGL_MAX_PBUFFER_PIXELS,
    EGL_NATIVE_RENDERABLE,
    EGL_NATIVE_VISUAL_ID,
    EGL_NATIVE_VISUAL_TYPE,
    EGL_SAMPLE_BUFFERS,
    EGL_SAMPLES,
    EGL_STENCIL_SIZE,
    EGL_SURFACE_TYPE,
    EGL_TRANSPARENT_TYPE,
    EGL_TRANSPARENT_RED_VALUE,
    EGL_TRANSPARENT_GREEN_VALUE,
    EGL_TRANSPARENT_BLUE_VALUE,
};

for (unsigned i = 0; i < sizeof(attrs) / sizeof(attrs[0]); ++i) {
    EGLint value = 0;
    if (eglGetConfigAttrib(dpy, config, attrs[i], &value)) {
        printf("attr 0x%04x = %d\n", attrs[i], value);
    } else {
        printf("attr 0x%04x failed: 0x%04x\n", attrs[i], eglGetError());
    }
}
```

Pratik Özet

- Bu fonksiyon config seçmez; seçilmiş config'i okur.
- Her çağrı tek attribute döndürür.
- EGL 1.0 uyumu için sadece Table 3.1 attribute'larını kullan.
- `EGL_SURFACE_TYPE` bitmask'tir; exact integer gibi yorumlama.

- `EGL_NATIVE_VISUAL_ID` platform-dependent olduğundan anlamı X11, GBM veya başka native platforma göre değişebilir.

EGL 1.0: `eglCreateWindowSurface`

```
EGLSurface eglCreateWindowSurface(EGLDisplay dpy,  
                                   EGLConfig config,  
                                   NativeWindowType win,  
                                   const EGLint *attrib_list);
```

`eglCreateWindowSurface` , önceden oluşturulmuş bir native window üzerinde ekrana çizim yapılabilen bir `EGLSurface` oluşturur.

EGL 1.0 standardı `NativeWindowType` nesnesinin gerçek türünü platforma bırakır. Bu projede X11 veya Wayland kullanılmadığı için native window rolünü Mesa/GBM tarafındaki `struct gbm_surface *` üstlenir.

Bu projedeki temel kullanım:

```
EGLSurface egl_surface = eglCreateWindowSurface(  
    egl_display,  
    egl_config,  
    (EGLNativeWindowType)gbm_surface,  
    NULL  
);
```

Mental Model

EGL 1.0 açısından `eglCreateWindowSurface` native window'u kendisi oluşturmaz. Daha önce native platform tarafından oluşturulmuş window nesnesinin üzerine bir EGL rendering surface oluşturur.

```

Native Platform
|
+-- Native Display
|
+-- Native Window
    |
    v
    eglCreateWindowSurface
    |
    v
    EGLSurface

```

Bu projedeki karşılığı:

```

/dev/dri/card*
|
v
DRM/KMS
|
v
gbm_device
|
v
gbm_surface
|
| NativeWindowType olarak EGL'e verilir
v
eglCreateWindowSurface
|
v
EGLSurface
|
v
OpenGL ES 2.0

```

Burada iki farklı surface kavramı vardır:

```

struct gbm_surface * -> Mesa/GBM native surface
EGLSurface           -> EGL rendering surface

```

`gbm_surface` , EGL'in bağlanacağı native platform nesnesidir. `EGLSurface` ise OpenGL ES context'inin çizim yapacağı EGL nesnesidir.

Parametreler

dpy

dpy , surface'in oluşturulacağı initialized EGLDisplay nesnesidir.

Değer	Sonuç
Geçerli ve initialized EGLDisplay	Diğer parametreler de uygunsa surface oluşturulabilir.
GBM tabanlı initialized EGLDisplay	Bu projede kullanılacak normal durumdur.

Bu projede display zinciri kabaca:

```

DRM file descriptor
    |
    v
gbm_create_device()
    |
    v
struct gbm_device *
    |
    v
EGLDisplay

```

Örnek:

```

EGLSurface egl_surface = eglCreateWindowSurface(
    egl_display,
    egl_config,
    (EGLNativeWindowType)gbm_surface,
    NULL
);

```

dpy çözünürlük, renk formatı veya buffer tipi seçmez. Hangi EGL display bağlantısında çalışılacağını belirtir.

Pratik kural:

```
dpy = GBM native platformuna bağlı ve eglInitialize() ile başlatılır
```

config

`config` , oluşturulacak surface ile kullanılacak framebuffer/pixel yapılandırmasını temsil eden `EGLConfig` nesnesidir.

Bu handle genellikle `eglChooseConfig` veya `eglGetConfigs` ile elde edilir. Window surface oluşturulabilmesi için config'in `EGL_SURFACE_TYPE` özelliğinde `EGL_WINDOW_BIT` bulunmalıdır.

Örnek seçim:

```
const EGLint config_attribs[] = {
    EGL_SURFACE_TYPE, EGL_WINDOW_BIT,
    EGL_RED_SIZE,      8,
    EGL_GREEN_SIZE,    8,
    EGL_BLUE_SIZE,     8,
    EGL_ALPHA_SIZE,    8,
    EGL_NONE
};

EGLConfig egl_config;
EGLint num_configs;

eglChooseConfig(
    egl_display,
    config_attribs,
    &egl_config,
    1,
    &num_configs
);
```

Sonra:

```
EGLSurface egl_surface = eglCreateWindowSurface(
    egl_display,
    egl_config,
    (EGLNativeWindowType)gbm_surface,
    NULL
);
```

Farklı `config` değerleri

`eglCreateWindowSurface` içine RGB değerleri doğrudan verilmez. Farklı özelliklere sahip `EGLConfig` handle'ları seçilir.

Örneğin:


```
Config A
R = 8
G = 8
B = 8
A = 8
Surface Type = EGL_WINDOW_BIT
```

ve:

```
Config B
R = 5
G = 6
B = 5
Surface Type = EGL_WINDOW_BIT
```

Sonra:

```
eglCreateWindowSurface(dpy, config_a, win, NULL);
eglCreateWindowSurface(dpy, config_b, win, NULL);
```

şeklinde farklı config'ler denenebilir.

Bu projede seçilen `EGLConfig` ile GBM surface'in pixel formatının da uyumlu olması gerekir.

Örneğin GBM tarafında:

```
gbm_surface_create(
    gbm,
    width,
    height,
    GBM_FORMAT_XRGB8888,
    GBM_BO_USE_SCANOUT | GBM_BO_USE_RENDERING
);
```

kullanılıyorsa native GBM surface ile seçilen EGL config'in uyumlu olması gerekir.

win

`win` , EGL 1.0 standardında platforma özgü native window handle'ıdır.

EGL 1.0, bu parametrenin gerçek C türünü tek başına belirlemez. Platform entegrasyonu hangi native window türünü kullanıyorsa o nesne buraya verilir.

Klasik örnekler:

```
X11      -> X11 Window
Wayland  -> Wayland surface
Windows  -> native window handle
```

Bu projede ise:

```
Mesa/GBM -> struct gbm_surface *
```

kullanılır.

Önce GBM surface oluşturulur:

```
struct gbm_surface *gbm_surface =
    gbm_surface_create(
        gbm,
        mode.hdisplay,
        mode.vdisplay,
        GBM_FORMAT_XRGB8888,
        GBM_BO_USE_SCANOUT | GBM_BO_USE_RENDERING
    );
```

Daha sonra:

```
EGLSurface egl_surface = eglCreateWindowSurface(
    egl_display,
    egl_config,
    (EGLNativeWindowType)gbm_surface,
    NULL
);
```

Burada:

```
win = (EGLNativeWindowType)gbm_surface
```

olur.

Farklı win değerleri

İki farklı GBM surface:

```
struct gbm_surface *gbm_surface_a;
struct gbm_surface *gbm_surface_b;
```

için:

```
EGLSurface surface_a = eglCreateWindowSurface(  
    egl_display,  
    egl_config,  
    (EGLNativeWindowType)gbm_surface_a,  
    NULL  
);  
  
EGLSurface surface_b = eglCreateWindowSurface(  
    egl_display,  
    egl_config,  
    (EGLNativeWindowType)gbm_surface_b,  
    NULL  
);
```

Sonuç:

```
gbm_surface_a -> EGLSurface A  
gbm_surface_b -> EGLSurface B
```

Yani `win` değişirse EGLSurface'in bağlı olduğu native surface değişir.

DRM mode ile boyut ilişkisi

Direct-to-display kullanımda GBM surface boyutu seçilen DRM mode ile eşleştirilebilir:

```
mode.hdisplay = 1920  
mode.vdisplay = 1080
```

```
gbm_surface_create(  
    gbm,  
    mode.hdisplay,  
    mode.vdisplay,  
    GBM_FORMAT_XRGB8888,  
    GBM_BO_USE_SCANOUT | GBM_BO_USE_RENDERING  
);
```

Bu 1920x1080 GBM surface daha sonra `win` olarak EGL'e verilir.

`attrib_list`

`attrib_list`, window surface attribute listesidir.

EGL 1.0 core standardında `eglCreateWindowSurface` için tanımlanmış bir window attribute'u yoktur. Bu nedenle normal kullanımlar:

```
NULL
```

veya:

```
const EGLint surface_attribs[] = {  
    EGL_NONE  
};
```

şeklindedir.

1. NULL

```
EGLSurface egl_surface = eglCreateWindowSurface(  
    egl_display,  
    egl_config,  
    (EGLNativeWindowType)gbm_surface,  
    NULL  
);
```

Sonuç:

Ek window surface attribute'u belirtilmez.

2. { EGL_NONE }

```
const EGLint surface_attribs[] = {  
    EGL_NONE  
};  
  
EGLSurface egl_surface = eglCreateWindowSurface(  
    egl_display,  
    egl_config,  
    (EGLNativeWindowType)gbm_surface,  
    surface_attribs  
);
```

`EGL_NONE` , attribute listesinin bittiğini belirtir.

EGL 1.0 core açısından:

```
NULL
```

ve:

```
{ EGL_NONE }
```

ek window attribute verilmediğini ifade eder.

Not: Platform extension'ları ek attribute'lar tanımlayabilir. Bunlar EGL 1.0 core davranışı değildir.

Geçerli Kombinasyonlar

1. GBM surface + `NULL`

```
EGLSurface egl_surface = eglCreateWindowSurface(  
    egl_display,  
    egl_config,  
    (EGLNativeWindowType)gbm_surface,  
    NULL  
);
```

```
dpy          = initialized GBM tabanlı EGLDisplay  
config       = EGL_WINDOW_BIT destekleyen uyumlu EGLConfig  
win          = gbm_surface  
attrib_list  = NULL
```

2. GBM surface + boş attribute listesi

```
const EGLint attrs[] = {  
    EGL_NONE  
};  
  
EGLSurface egl_surface = eglCreateWindowSurface(  
    egl_display,  
    egl_config,  
    (EGLNativeWindowType)gbm_surface,  
    attrs  
);
```

3. Farklı config

```
EGLSurface surface_a = eglCreateWindowSurface(  
    egl_display,  
    config_a,  
    (EGLNativeWindowType)gbm_surface,  
    NULL  
);
```

```
EGLSurface surface_b = eglCreateWindowSurface(  
    egl_display,  
    config_b,  
    (EGLNativeWindowType)gbm_surface,  
    NULL  
);
```

4. Farklı native GBM surface

```
EGLSurface surface_a = eglCreateWindowSurface(  
    egl_display,  
    egl_config,  
    (EGLNativeWindowType)gbm_surface_a,  
    NULL  
);
```

```
EGLSurface surface_b = eglCreateWindowSurface(  
    egl_display,  
    egl_config,  
    (EGLNativeWindowType)gbm_surface_b,  
    NULL  
);
```

Parametre Matrisi

dpy	config	win	attrib_list	Sonuç
GBM tabanlı initialized display	Uyumlu window config	gbm_surface	NULL	Bu proje için temel kullanım

dpy	config	win	attrib_list	Sonuç
GBM tabanlı initialized display	Uyumlu window config	gbm_surface	{ EGL_NONE }	Geçerli boş attribute listesi
Aynı display	Config A	Aynı GBM surface	NULL	Config A kullanılır
Aynı display	Config B	Aynı GBM surface	NULL	Config B kullanılır
Aynı display	Aynı config	GBM surface A	NULL	Surface A'ya bağlanır
Aynı display	Aynı config	GBM surface B	NULL	Surface B'ye bağlanır

Dönüş Değeri

Fonksiyonun dönüş tipi:

EGLSurface

Başarılı olduğunda oluşturulan surface handle'ını döndürür.

Başarısız olduğunda:

EGL_NO_SURFACE

döndürür.

Örnek kontrol:

```
EGLSurface egl_surface =
    eglCreateWindowSurface(
        egl_display,
        egl_config,
        (EGLNativeWindowType)gbm_surface,
        NULL
    );

if (egl_surface == EGL_NO_SURFACE) {
    EGLint err = eglGetError();
}
```

EGL 1.0 Hata Kodları

Hata	Ne zaman
EGL_BAD_MATCH	win özellikleri config ile uyuşmuyorsa veya config window rendering desteklemiyorsa.
EGL_BAD_CONFIG	config geçerli bir EGLConfig değilse.
EGL_BAD_NATIVE_WINDOW	win geçerli bir native window handle değilse.
EGL_BAD_ALLOC	Aynı native window ile daha önce bir EGLConfig ilişkilendirilmişse veya yeni surface için gerekli kaynaklar ayrılamıyorsa.

Window surface için config'in:

```
EGL_SURFACE_TYPE
```

özelliğinin:

```
EGL_WINDOW_BIT
```

içermesi gerekir.

GBM ile EGL 1.0 Arasındaki Sınır

EGL 1.0 core standardı şunları tanımlar:


```
EGLDisplay
EGLConfig
NativeWindowType kavramı
EGLSurface
eglCreateWindowSurface()
```

GBM, EGL 1.0 core standardının parçası değildir.

Bu projede Mesa/GBM entegrasyonu native platform tarafını sağlar:

```
EGL 1.0:
"Platform bana bir NativeWindowType sağlayacak."
      |
      v
Mesa / GBM:
Native window = struct gbm_surface *
```

Bu nedenle kullanılan yapı:

```
EGL 1.0 API
+
Mesa/GBM native platform entegrasyonu
+
DRM/KMS display yönetimi
```

şeklindedir.

`eglCreateWindowSurface` fonksiyonunun kendisi değiştirilmez. Değişen nokta, native platformun X11/Wayland yerine GBM olmasıdır.

Direct-to-Display Akışındaki Yeri

```
open("/dev/dri/card*")
|
v
drmModeGetResources()
|
v
Connector / Encoder / CRTC / Mode
|
v
gbm_create_device()
|
v
gbm_surface_create()
|
v
EGLDisplay
|
v
eglInitialize()
|
v
eglChooseConfig()
|
v
eglCreateWindowSurface()
|
v
eglCreateContext()
|
v
eglMakeCurrent()
|
v
OpenGL ES 2.0 Render
|
v
eglSwapBuffers()
|
v
gbm_surface_lock_front_buffer()
|
v
GBM BO
```

```
    |  
    v  
DRM Framebuffer  
    |  
    v  
drmModeSetCrtc() / drmModePageFlip()  
    |  
    v  
Physical Monitor
```

`eglCreateWindowSurface` monitör connector'ını, CRTC'yi veya display mode'u seçmez. Bu görevler DRM/KMS tarafındadır.

Minimal Doğru Kullanım

```
/* DRM device daha önce açılmıştır. */
int drm_fd = /* open("/dev/dri/card0", ...) */;

/* GBM device */
struct gbm_device *gbm = gbm_create_device(drm_fd);

/* DRM mode'dan alınan boyutlar */
uint32_t width  = mode.hdisplay;
uint32_t height = mode.vdisplay;

/* Native GBM surface */
struct gbm_surface *gbm_surface =
    gbm_surface_create(
        gbm,
        width,
        height,
        GBM_FORMAT_XRGB8888,
        GBM_BO_USE_SCANOUT | GBM_BO_USE_RENDERING
    );

/* Daha önce GBM platformundan elde edilmiş ve initialize edilmiş EGLDisplay
EGLDisplay egl_display = /* ... */;

/* eglChooseConfig ile seçilmiş uyumlu config */
EGLConfig egl_config = /* ... */;

EGLSurface egl_surface =
    eglCreateWindowSurface(
        egl_display,
        egl_config,
        (EGLNativeWindowType)gbm_surface,
        NULL
    );

if (egl_surface == EGL_NO_SURFACE) {
    EGLint err = eglGetError();
}
```

Parametre Değiştirme Örnekleri

1. attrib_list = NULL

```
EGLSurface surface = eglCreateWindowSurface(  
    egl_display,  
    egl_config,  
    (EGLNativeWindowType)gbm_surface,  
    NULL  
);
```

Değiştirilen parametre: `attrib_list`

Değer: `NULL`

Sonuç: Ek EGL 1.0 window surface attribute'u belirtilmez.

2. `attrib_list = { EGL_NONE }`

```
const EGLint attrs[] = {  
    EGL_NONE  
};  
  
EGLSurface surface = eglCreateWindowSurface(  
    egl_display,  
    egl_config,  
    (EGLNativeWindowType)gbm_surface,  
    attrs  
);
```

Değiştirilen parametre: `attrib_list`

Değer: boş attribute listesi

Sonuç: Ek EGL 1.0 window surface attribute'u belirtilmez.

3. `config` değiştirme

```
EGLSurface surface_a = eglCreateWindowSurface(  
    egl_display,  
    config_a,  
    (EGLNativeWindowType)gbm_surface,  
    NULL  
);  
  
EGLSurface surface_b = eglCreateWindowSurface(  
    egl_display,  
    config_b,  
    (EGLNativeWindowType)gbm_surface,  
    NULL  
);
```

Değiştirilen parametre: config

Sonuç: Surface için kullanılan EGL framebuffer/pixel yapılandırması:

4. win değiştirme

```
EGLSurface surface_a = eglCreateWindowSurface(  
    egl_display,  
    egl_config,  
    (EGLNativeWindowType)gbm_surface_a,  
    NULL  
);  
  
EGLSurface surface_b = eglCreateWindowSurface(  
    egl_display,  
    egl_config,  
    (EGLNativeWindowType)gbm_surface_b,  
    NULL  
);
```

Değiştirilen parametre: win

surface_a -> gbm_surface_a

surface_b -> gbm_surface_b

EGL 1.0 İçin Pratik Özet

- `eglCreateWindowSurface` , mevcut bir native window üzerinde on-screen

`EGLSurface` oluşturur.

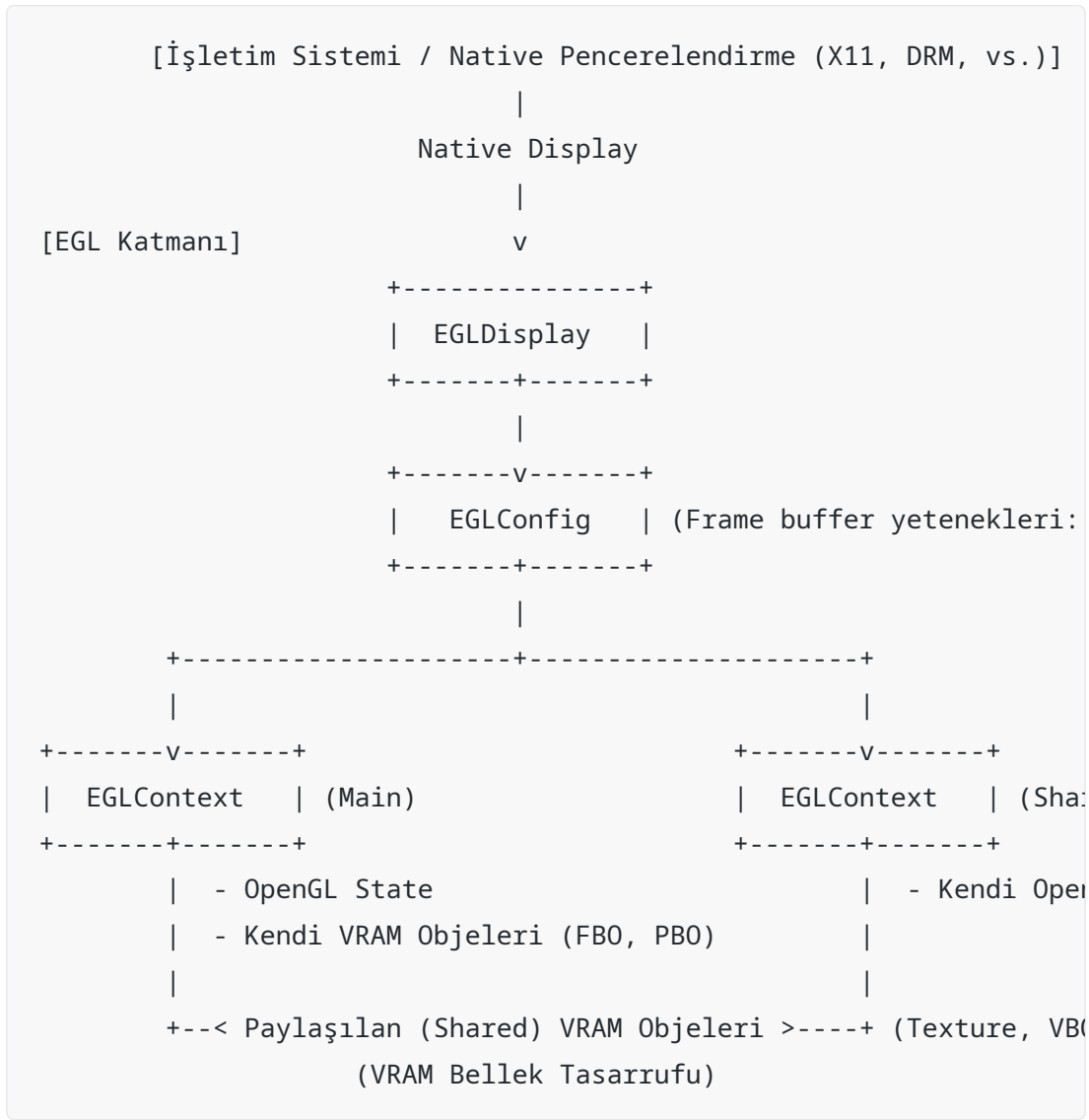
- Fonksiyon native window'u kendisi oluşturmaz.
- `dpy` , initialized `EGLDisplay` nesnesidir.
- `config` , surface'in EGL framebuffer/pixel yapılandırmasını belirler.
- `config` , window rendering için `EGL_WINDOW_BIT` desteklemelidir.
- `win` , platformun sağladığı native window handle'ıdır.
- Bu projede `win` , Mesa/GBM tarafından oluşturulan `struct gbm_surface *` nesnesidir.
- `attrib_list` , EGL 1.0 core'da normal olarak `NULL` veya `{ EGL_NONE }` kullanılır.
- Başarılı çağrı `EGLSurface` , başarısız çağrı `EGL_NO_SURFACE` döndürür.
- GBM, EGL 1.0 core standardının parçası değildir; native platform entegrasyonunu sağlar.
- DRM/KMS fiziksel display ve scan-out işlemlerini yönetir; EGL/OpenGL ES rendering tarafını yönetir.
-

EGL 1.0: `eglCreateContext`

```
EGLContext eglCreateContext(EGLDisplay dpy,  
                             EGLConfig config,  
                             EGLContext share_context,  
                             const EGLint *attrib_list);
```

`eglCreateContext` , OpenGL ES veya diğer Khronos API'lerinin komutlarını çalıştıracağı durumu (state machine) ve bellek alanını (rendering context) temsil eden soyut bir bağlam oluşturur.

Mental Model



Parametreler

`dp` (**EGLDisplay**)

Bağlantı kurulan fiziksel veya sanal ekranın (display) tanıtıcısıdır.

Değer	Sonuç
Geçerli ve initialize edilmiş <code>EGLDisplay</code>	İşlem diğer parametreler de doğruysa devam eder.
<code>EGL_NO_DISPLAY</code> veya sahte display handle	Başarısız. <code>EGL_BAD_DISPLAY</code> hatası döner.
<code>eglInitialize</code> çağrılmamış display	Başarısız. <code>EGL_NOT_INITIALIZED</code> hatası döner.

`config` (**EGLConfig**)

Oluşturulacak bağlamanın renk, derinlik (depth) ve stencil tampon gereksinimlerini belirleyen donanım konfigürasyonudur.

Değer	Sonuç
<code>dpy</code> üzerinden <code>eglChooseConfig</code> ile alınmış geçerli <code>EGLConfig</code>	Başarılı. Context bu formatta render yapmak üzere ayarlanır.
Geçersiz veya uyuşmaz config handle	Başarısız. <code>EGL_BAD_CONFIG</code> hatası döner.

`share_context` (EGLContext)

Doku (texture) ve köşe tamponu (VBO) gibi VRAM nesnelerinin paylaşılacağı mevcut bir context. Güvenlik kritik sistemlerde bellek tasarrufu için hayati öneme sahiptir.

Değer	Sonuç
<code>EGL_NO_CONTEXT</code>	İzolasyon. Context hiçbir VRAM verisini paylaşmaz, izole çalışır.
Geçerli bir <code>EGLContext</code> handle	Paylaşım. İki context texture gibi objelere ortak erişim sağlar. State'ler (viewport vb.) ayrı kalır.
Geçersiz handle veya farklı <code>dpy</code> 'ye ait context	Başarısız. <code>EGL_BAD_CONTEXT</code> veya <code>EGL_BAD_MATCH</code> hatası döner.

`attrib_list` (const EGLint *)

Context oluşturulurken istenen ekstra özellikleri (API sürümü vb.) belirten anahtar-değer (key-value) çifti listesidir.

Değer	Sonuç
<code>NULL</code> veya <code>{ EGL_NONE }</code>	EGL 1.0 standart kullanımı. Varsayılan (default) context (genellikle GLES 1.x) oluşturulur.
Geçerli attribute listesi (Örn: GLES 2.0 talebi)	İlgili gereksinimleri karşılayan bağlam oluşturulur.
Geçersiz/Desteklenmeyen attribute	Başarısız. <code>EGL_BAD_ATTRIBUTE</code> hatası döner.

Geçerli Attribute Listesi

`attrib_list`, { anahtar, değer, ..., `EGL_NONE` } formatında sonlanan bir dizidir.

Attribute	Tip	Anlam
<code>EGL_CONTEXT_CLIENT_VERSION</code>	<code>EGLint</code>	İstenen OpenGL ES sürümü (Örn: 1 veya 2). EGL 1.0 spesifikasyonunda standart olarak attribute alınmaz (Bölüm 3.6.1), ancak EGL 1.3 ve sonrası uzantılarla bu parametre GLES 2.x/3.x kullanımı için kritik hale gelmiştir.
<code>EGL_NONE</code>	<code>EGLint</code>	Liste sonlandırıcı belirteç. Zorunludur.

Not: `eglCreateContext` doğrudan buffer tiplerine (single buffered pixmap veya back buffered window) bağımlı değildir; bu uyumluluk yüzey (surface) oluşturulurken (`eglCreateWindowSurface`) ve bağlam yüzeye bağlanırken (`eglMakeCurrent`) denetlenir.

Ayrıntılar ve Yaşam Döngüsü

- **Thread Güvenliği (Thread Safety):** `EGLContext` aynı anda (concurrently) sadece bir iş parçacığında (thread) aktif (`current`) olabilir. Başka bir thread bu context'i `eglMakeCurrent` ile aktif etmek isterse, mevcut thread'in önce bağlamı serbest bırakması (unbind) gerekir. EGL spesifikasyonu, bir context'in birden fazla thread'de kullanılmasını yasaklar (`EGL_BAD_ACCESS` döner).
- **Yaşam Döngüsü:** Context oluşturulduğunda bellek tahsisi yapılır ancak ekrana çizim yapamaz. Çizim için `eglMakeCurrent` şarttır. Yok edilmesi ise `eglDestroyContext` ile yapılır.
- **Eşzamanlama (Synchronization):** `share_context` ile VRAM paylaşımı yapıldığında, OpenGL ES komutları otomatik olarak senkronize olmaz. İki farklı thread, paylaşılan bir dokuya (texture) aynı anda yazmaya/okumaya çalışırsa Undefined Behavior (tanımsız davranış) oluşur. Bu durumu engellemek için `glFinish()`, `eglWaitGL()` veya `eglWaitNative()` gibi açık eşzamanlama bariyerleri kullanılmalıdır.

Hata Matrisi

Khronos spesifikasyonu (Bölüm 3.6.1) kuralı gereği: Fonksiyon başarısız olduğunda state machine'de hiçbir değişiklik olmaz, VRAM veya RAM sızıntısı yaşanmaz (No side effects). Hata durumunda `EGL_NO_CONTEXT` döner. Hatayı okumak için `eglGetError()` çağrılmalıdır.

Durum	Sonuç (EGL Hata Kodu)
Geçerli parametreler ve yeterli donanım/ bellek	Başarılı (Geçerli <code>EGLContext</code> döner)
<code>dpy</code> geçerli bir display değilse	<code>EGL_BAD_DISPLAY</code>
<code>dpy</code> EGL ile initialize edilmemişse	<code>EGL_NOT_INITIALIZED</code>
<code>config</code> geçersiz bir EGL konfigürasyonu ise	<code>EGL_BAD_CONFIG</code>
<code>share_context</code> geçersizse (<code>EGL_NO_CONTEXT</code> değilse)	<code>EGL_BAD_CONTEXT</code>
Context'ler paylaşılamıyorsa (uyuşmaz API veya donanım kısıtlaması)	<code>EGL_BAD_MATCH</code>
<code>attrib_list</code> geçersiz bir attribute içeriyorsa	<code>EGL_BAD_ATTRIBUTE</code>
İşletim sistemi veya GPU belleğinde yer kalmadıysa	<code>EGL_BAD_ALLOC</code>

Güvenli Kullanım Örneği

```
#include <EGL/egl.h>
#include <stdio.h>
#include <stdlib.h>

// Güvenlik kritik sistemler (Avionics) için Robust Context Oluşturucu
EGLContext CreateSecureContext(EGLDisplay dpy, EGLConfig config, EGLint* attrib_list)
{
    // EGL 1.0 uyumlu null-terminated liste
    const EGLint attrib_list[] = {
        EGL_NONE // GLES 1.x / EGL 1.0 Default
    };

    // Context oluştur (Thread güvenli ve state-protected çağrı)
    EGLContext context = eglCreateContext(dpy, config, shared_ctx, attrib_list);

    // Error checking (Hata kontrolü) - Zero Side Effect kuralı ile
    if (context == EGL_NO_CONTEXT) {
        EGLint err = eglGetError();
        switch (err) {
            case EGL_BAD_DISPLAY:
                fprintf(stderr, "Kritik Hata: Gecersiz Display Handle\n");
                break;
            case EGL_NOT_INITIALIZED:
                fprintf(stderr, "Kritik Hata: EGL sistemi baslatılmamış\n");
                break;
            case EGL_BAD_CONFIG:
                fprintf(stderr, "Hata: Donanım konfigürasyonu geçersiz\n");
                break;
            case EGL_BAD_ALLOC:
                fprintf(stderr, "OOM (Out of Memory): GPU veya Sistem Belleği\n");
                // Aviyonik sistemlerde bu durum watchdog timer veya reset ile
                break;
            case EGL_BAD_MATCH:
                fprintf(stderr, "Hata: Paylasimli context konfigürasyonu\n");
                break;
            case EGL_BAD_ATTRIBUTE:
                fprintf(stderr, "Hata: Desteklenmeyen EGL_ATTRIBUTE\n");
                break;
            case EGL_BAD_CONTEXT:
                fprintf(stderr, "Hata: Paylasilmak istenen kaynak\n");
                break;
            default:
                fprintf(stderr, "Bilinmeyen EGL hatasi: 0x%04x\n", err);
                break;
        }
    }
}
```

```
    }  
    return EGL_NO_CONTEXT;  
}  
  
// Başarılı: Bellek tahsisi ve state machine hazır.  
return context;  
}
```

Pratik Özet

- **State Yönetimi:** `eglCreateContext` , OpenGL'in çalışacağı boş beyni oluşturur; ancak gözleri (yüzey) ve elleri (current thread) yoktur. `eglMakeCurrent` çağırmadan OpenGL ES komutları işlenemez.
- **Bellek Optimizasyonu (Shared Context):** Uçuş paneli gibi çok ekranlı (PFD, ND, EICAS) sistemlerde her ekrana ayrı bağlam (context) açmak yerine, VRAM dokularını (harita verileri, simgeler, fontlar) `share_context` ile paylaşmak muazzam bellek tasarrufu sağlar.
- **Güvenlik Kritik İzolasyon:** Hata oluştuğunda sistem state'i kesinlikle bozulmaz (`Zero side-effect`). Bu durum, DO-178C gibi havacılık standartlarında öngörülebilir deterministik davranışların sağlanabilmesi için idealdir.
- **Thread Kısıtlaması:** Context bir kez bir iş parçasına (thread) `eglMakeCurrent` ile bağlandığında (bind edildiğinde), bağ bitene kadar başka hiçbir thread o bağlama dokunamaz. İhlal edilmesi fatal hata ve `EGL_BAD_ACCESS` doğurur.
- **Hata Kontrolü:** Fonksiyonun dönüş değeri daima `EGL_NO_CONTEXT` sabiti ile kıyaslanmalı, `NULL` (0) kontrolü yapılmamalıdır. Hata alınması durumunda yalnızca `eglGetError()` durumu aydınlatılabilir.

EGL 1.0: `eglMakeCurrent`

```
EGLBoolean eglMakeCurrent(EGLDisplay dpy,  
                           EGLSurface draw,  
                           EGLSurface read,  
                           EGLContext ctx);
```

`eglMakeCurrent` , bir `EGLContext` nesnesini çağıran thread'in current rendering context'i yapar. Aynı çağrıda iki surface bağlanır:

- `draw` : OpenGL ES çizim komutlarının yazdığı framebuffer.

- `read : glReadPixels` gibi okuma komutlarının okuduğu framebuffer.

En yaygın kullanımda `draw` ve `read` aynı surface'tir:

```
eglMakeCurrent(dpy, surface, surface, ctx);
```

Mental Model

EGL 1.0 açısından current context thread-local bir durumdur:

```
Thread
|
+-- current EGLDisplay
+-- current EGLContext
+-- current draw EGLSurface
+-- current read EGLSurface
```

`eglMakeCurrent` bu dördlüyü değiştirir. OpenGL ES komutları doğrudan `EGLContext` handle'ına parametre olarak verilmez; komutlar çağıran thread'in current context'i üzerinden çalışır.

Parametreler

dpy

`dpy`, context ve surface nesnelerinin ait olduğu initialized `EGLDisplay` olmalıdır.

Değer	Sonuç
Geçerli ve initialized <code>EGLDisplay</code>	Diğer parametreler de geçerliyse çağrı başarılıdır.
<code>EGL_NO_DISPLAY</code>	Başarısız. Genel EGL hata modeliyle <code>EGL_BAD_DISPLAY</code> beklenir.
Geçersiz display handle	Başarısız. Genel EGL hata modeliyle <code>EGL_BAD_DISPLAY</code> beklenir.
Initialize edilmemiş display	Başarısız. Genel EGL hata modeliyle <code>EGL_NOT_INITIALIZED</code> beklenir.

Pratik kural:

```
EGLDisplay dpy = eglGetDisplay(native_display);
eglInitialize(dpy, &major, &minor);
```

`eglInitialize` başarılı olmadan `eglMakeCurrent` çağırma.

draw

`draw` , çizim hedefidir.

Değer	Sonuç
<code>ctx</code> ile uyumlu geçerli <code>EGLSurface</code>	Geçerli. OpenGL ES draw komutları buraya yazar.
<code>read</code> ile aynı surface	Geçerli ve normal kullanım.
<code>read</code> 'den farklı ama uyumlu surface	Geçerli.
<code>EGL_NO_SURFACE</code> ve <code>ctx == EGL_NO_CONTEXT</code> ve <code>read == EGL_NO_SURFACE</code>	Geçerli. Current context release edilir.
<code>EGL_NO_SURFACE</code> ve <code>ctx != EGL_NO_CONTEXT</code>	Başarısız, <code>EGL_BAD_MATCH</code> .
Geçersiz surface	Başarısız, <code>EGL_BAD_SURFACE</code> .
Yok edilmiş surface	Başarısız veya sonraki framebuffer davranışı tanımsız.
Native window'u geçersiz window surface	Başarısız, <code>EGL_BAD_NATIVE_WINDOW</code> .
<code>ctx</code> ile uyumsuz surface	Başarısız, <code>EGL_BAD_MATCH</code> .
Başka thread'de başka context'e bağlı surface	Başarısız, <code>EGL_BAD_ACCESS</code> .

read

`read` , framebuffer okuma kaynağıdır.

Değer	Sonuç
<code>ctx</code> ile uyumlu geçerli <code>EGLSurface</code>	Geçerli. <code>glReadPixels</code> buradan okur.

Değer	Sonuç
draw ile aynı surface	Geçerli ve normal kullanım.
draw 'dan farklı ama uyumlu surface	Geçerli.
EGL_NO_SURFACE ve ctx == EGL_NO_CONTEXT ve draw == EGL_NO_SURFACE	Geçerli. Current context release edilir.
EGL_NO_SURFACE ve ctx != EGL_NO_CONTEXT	Başarısız, EGL_BAD_MATCH .
Geçersiz surface	Başarısız, EGL_BAD_SURFACE .
Yok edilmiş surface	Başarısız veya readback sonucu tanımsız.
Native window'u geçersiz window surface	Başarısız, EGL_BAD_NATIVE_WINDOW .
ctx ile uyumsuz surface	Başarısız, EGL_BAD_MATCH .
Başka thread'de başka context'e bağlı surface	Başarısız, EGL_BAD_ACCESS .

ctx

ctx , current yapılacak rendering context'tir.

Değer	Sonuç
Geçerli EGLContext	Çağırın thread'in current context'i olur.
EGL_NO_CONTEXT , draw == EGL_NO_SURFACE , read == EGL_NO_SURFACE	Geçerli. Current context kaldırılır.
EGL_NO_CONTEXT , ama draw veya read gerçek surface	Başarısız, EGL_BAD_MATCH .
Geçersiz context	Başarısız, EGL_BAD_CONTEXT .
Başka thread'de current olan context	Başarısız, EGL_BAD_ACCESS .

Değer	Sonuç
Surface'lerle uyumsuz context	Başarısız, EGL_BAD_MATCH .

Geçerli Kombinasyonlar

1. Normal bind

```
eglMakeCurrent(dpy, surface, surface, ctx);
```

Sonuç:

```
Thread current context = ctx
Thread draw surface    = surface
Thread read surface    = surface
```

2. Ayrı draw/read surface

```
eglMakeCurrent(dpy, draw_surface, read_surface, ctx);
```

Sonuç:

```
Draw commands -> draw_surface
Read commands -> read_surface
```

Bu kullanım daha nadirdir ama EGL 1.0 tarafından desteklenir.

3. Current context'i bırakma

```
eglMakeCurrent(dpy, EGL_NO_SURFACE, EGL_NO_SURFACE, EGL_NO_CONTEXT)
```

Bu, EGL 1.0'da current context'i release etmenin doğru biçimidir.

Geçersiz Kombinasyon Matrisi

draw	read	ctx	Sonuç
surface	surface	context	Geçerli, uyumlusalar.
surface A	surface B	context	Geçerli, ikisi de uyumluysa.

draw	read	ctx	Sonuç
EGL_NO_SURFACE	EGL_NO_SURFACE	EGL_NO_CONTEXT	Geçerli release çağırısı.
surface	surface	EGL_NO_CONTEXT	EGL_BAD_MATCH
EGL_NO_SURFACE	surface	context	EGL_BAD_MATCH
surface	EGL_NO_SURFACE	context	EGL_BAD_MATCH
EGL_NO_SURFACE	EGL_NO_SURFACE	context	EGL_BAD_MATCH
invalid surface	surface	context	EGL_BAD_SURFACE
surface	invalid surface	context	EGL_BAD_SURFACE
surface	surface	invalid context	EGL_BAD_CONTEXT
incompatible surface	surface	context	EGL_BAD_MATCH
surface	incompatible surface	context	EGL_BAD_MATCH

Hata Kodları

Hata	Ne zaman
EGL_BAD_MATCH	Surface/context uyumsuzsa; EGL_NO_CONTEXT / EGL_NO_SURFACE kombinasyonu yanlışsa; draw/read aynı anda belleğe sığamıyorsa.
EGL_BAD_ACCESS	ctx başka thread'de current ise; draw veya read başka thread'de bir context'e bağlıysa.
EGL_BAD_CONTEXT	ctx geçerli EGL context değilse.
EGL_BAD_SURFACE	draw veya read geçerli EGL surface değilse.
EGL_BAD_NATIVE_WINDOW	Surface'in altında yatan native window artık geçerli değilse.

Hata	Ne zaman
EGL_BAD_CURRENT_SURFACE	Önceki current context'in flush edilmemiş komutları varsa ve önceki surface artık geçerli değilse.
EGL_BAD_ALLOC	Draw/read için gerekli ancillary buffer'lar ayrılamazsa.

State Değişimi

Başarılı çağrı öncesi:

```
Thread current context = old_ctx
Thread draw surface    = old_draw
Thread read surface    = old_read
```

Başarılı `eglMakeCurrent(dpy, draw, read, ctx)` sonrası:

```
old_ctx flush edilir
old_ctx artık current değildir

Thread current context = ctx
Thread draw surface    = draw
Thread read surface    = read
```

Başarılı release sonrası:

```
eglMakeCurrent(dpy, EGL_NO_SURFACE, EGL_NO_SURFACE, EGL_NO_CONTEXT)

Thread current context = EGL_NO_CONTEXT
Thread draw surface    = EGL_NO_SURFACE
Thread read surface    = EGL_NO_SURFACE
```

Viewport ve Scissor Notu

EGL 1.0 spec'ine göre bir context ilk kez current yapıldığında viewport ve scissor boyutları draw surface boyutuna ayarlanır:

```
glViewport(0, 0, draw_width, draw_height)
glScissor(0, 0, draw_width, draw_height)
```

Bu yalnızca context'in ilk current yapılma anı için önemlidir. Sonraki bind

işlemlerinde viewport/scissor'ın otomatik güncelleneceğini varsayma.

Destroy Sonrası Davranış

`eglMakeCurrent` başarılı olduktan sonra:

- `draw` destroy edilirse render komutları işlenebilir ama framebuffer state tanımsız olur.
- `read` destroy edilirse `glReadPixels` gibi okuma sonuçları tanımsız olur.
- Native window/pixmap destroy edilirse de ilgili surface için aynı mantık geçerlidir.

Bu yüzden güvenli kapanış sırası genelde şudur:

```
eglMakeCurrent(dpy, EGL_NO_SURFACE, EGL_NO_SURFACE, EGL_NO_CONTEXT);
eglDestroySurface(dpy, surface);
eglDestroyContext(dpy, ctx);
eglTerminate(dpy);
```

Minimal Doğru Kullanım

```
EGLDisplay dpy = eglGetDisplay(native_display);
eglInitialize(dpy, &major, &minor);

EGLConfig config = /* eglChooseConfig ile alınmış config */;
EGLSurface surface = /* eglCreateWindowSurface veya eglCreatePbuffer */;
EGLContext ctx = eglCreateContext(dpy, config, EGL_NO_CONTEXT, NULL);

if (!eglMakeCurrent(dpy, surface, surface, ctx)) {
    EGLint err = eglGetError();
}

/* OpenGL ES komutları */

eglMakeCurrent(dpy, EGL_NO_SURFACE, EGL_NO_SURFACE, EGL_NO_CONTEXT);
```

Bilerek Hata Üretme Örnekleri

```
/* EGL_BAD_MATCH: context yok ama surface var */
eglMakeCurrent(dpy, surface, surface, EGL_NO_CONTEXT);

/* EGL_BAD_MATCH: context var ama draw yok */
eglMakeCurrent(dpy, EGL_NO_SURFACE, surface, ctx);

/* EGL_BAD_MATCH: context var ama read yok */
eglMakeCurrent(dpy, surface, EGL_NO_SURFACE, ctx);

/* Doğru release */
eglMakeCurrent(dpy, EGL_NO_SURFACE, EGL_NO_SURFACE, EGL_NO_CONTEXT
```

EGL 1.0 İçin Pratik Özet

- `eglMakeCurrent` thread-local current context'i değiştirir.
- OpenGL ES komutlarının hangi context/surface üzerinde çalışacağını bu çağrı belirler.
- `draw` çizim hedefidir, `read` okuma hedefidir.
- `EGL_NO_CONTEXT` sadece iki surface de `EGL_NO_SURFACE` ise geçerlidir.
- Context veya surface başka thread'de bağlıysa `EGL_BAD_ACCESS` beklenir.
- Surface/context format ve display açısından uyumsuzsa `EGL_BAD_MATCH` beklenir.

EGL 1.0: `eglGetCurrentDisplay`

```
EGLDisplay eglGetCurrentDisplay(void);
```

`eglGetCurrentDisplay` , çağıran thread üzerinde current durumda olan EGL context ile ilişkili `EGLDisplay` handle'ını döndürür.

Kısa özet:

- Parametre almaz.
- Current context varsa ilişkili `EGLDisplay` döner.
- Current context yoksa `EGL_NO_DISPLAY` döner.
- Display'in yalnızca initialize edilmiş olması yeterli değildir.
- Davranışı parametreye değil current EGL state'e bağlıdır.

Mental Model

```
Thread
|
+-- current EGLContext
|
+-- current draw EGLSurface
|
+-- current read EGLSurface
|
+-- current EGLDisplay
```

`eglGetCurrentDisplay` , bu thread-local current state içindeki display'i sorgular.

Parametreler

Fonksiyon parametre almaz:

```
eglGetCurrentDisplay(void);
```

Bu nedenle görevdeki "parametreleri değiştirerek davranışı gözlemleme" yaklaşımı burada uygulanamaz.

Bunun yerine current EGL state değiştirildi.

Deney State'leri

```
State 1
Current context yok
|
v
eglGetCurrentDisplay()
|
v
EGL_NO_DISPLAY
```

```
State 2
Display initialize edildi
Context/surface mevcut
Ama eglMakeCurrent çağrılmadı
|
v
EGL_NO_DISPLAY
```

```
State 3
eglMakeCurrent(...)
    |
    v
Context current
    |
    v
eglGetCurrentDisplay()
    |
    v
İlgili EGLDisplay
```

```
State 4
Current context release edildi
    |
    v
eglGetCurrentDisplay()
    |
    v
EGL_NO_DISPLAY
```

Yardımcı Fonksiyonlar

Bu deneyi oluşturmak için şu yardımcı fonksiyonlar kullanıldı:

```
eglGetDisplay
eglInitialize
eglChooseConfig
eglCreatePbufferSurface
eglBindAPI
eglCreateContext
eglMakeCurrent
eglDestroyContext
eglDestroySurface
eglTerminate
```

Bunlar `eglGetCurrentDisplay` fonksiyonunun parçası değildir. Ayrıntıları için kitabın ilgili bölümlerine bakınız.

Özellikle current state'i değiştiren `eglMakeCurrent` için ilgili bölüme bakınız.

Neden Pbuffer Kullanıldı?

Test için görünür pencere gerekmiyordu.

Bu nedenle 64x64 pbuffer oluşturuldu:

```
const EGLint pbuffer_attributes[] = {
    EGL_WIDTH, 64,
    EGL_HEIGHT, 64,
    EGL_NONE
};
```

Amaç, context'in bağlanabileceği geçerli bir surface sağlamaktır.

Test Altyapısında İlk Context Hatası

İlk context oluşturma denemesi:

```
eglCreateContext(
    display,
    config,
    EGL_NO_CONTEXT,
    NULL
);
```

ile yapıldı ve:

```
EGL_BAD_ALLOC (0x3003)
```

alındı.

Bu `eglGetCurrentDisplay` hatası değildi; yardımcı context oluşturma aşamasına aitti.

Daha sonra test için açıkça OpenGL ES 2 context istendi:

```
eglBindAPI(EGL_OPENGL_ES_API);

const EGLint context_attributes[] = {
    EGL_CONTEXT_CLIENT_VERSION, 2,
    EGL_NONE
};
```

Config seçiminde de:

```
EGL_RENDERABLE_TYPE, EGL_OPENGL_ES2_BIT
```


eklendi.

EGL_RENDERABLE_TYPE ve EGL_CONTEXT_CLIENT_VERSION , EGL 1.0 config attribute setinin parçası değildir. Burada yalnızca modern Mesa ortamında test için gerekli yardımcı OpenGL ES 2 context'ini oluşturmak amacıyla kullanılmıştır.

Test 1 — Current Context Yok

Programın başında:

```
EGLDisplay current = eglGetCurrentDisplay();
```

Gerçek çıktı:

```
eglGetCurrentDisplay() : (nil)
State                  : EGL_NO_DISPLAY
```

Bu durum normaldir; henüz current context yoktur.

Test 2 — Display Initialize Edildi, Context Current Değil

Display:

```
eglGetDisplay(EGL_DEFAULT_DISPLAY);
eglInitialize(display, &major, &minor);
```

ile hazırlandı.

Gerçek bilgi:

```
Runtime version : 1.5
EGLDisplay      : 0x59fd88c23b50
```

Ancak `eglMakeCurrent` çağrılmadan:

```
eglGetCurrentDisplay() : (nil)
State                  : EGL_NO_DISPLAY
```

elde edildi.

Bu deney:

```
Initialized EGLDisplay
    !=
Current EGLDisplay
```

ayrımını doğrudan gösterir.

Test 3 — Context Current

```
eglMakeCurrent(
    display,
    surface,
    surface,
    context
);
```

sonrasında:

```
EGLDisplay current = eglGetCurrentDisplay();
```

Gerçek çıktı:

```
eglGetCurrentDisplay() : 0x59fd88c23b50
State                  : Valid EGLDisplay
Expected EGLDisplay    : 0x59fd88c23b50
Comparison              : SAME HANDLE
```

Current context'in ilişkili olduğu display döndü.

Test 4 — Current Context Release

Current context şu şekilde bırakıldı:

```
eglMakeCurrent(
    display,
    EGL_NO_SURFACE,
    EGL_NO_SURFACE,
    EGL_NO_CONTEXT
);
```

Sonuç:

```
eglGetCurrentDisplay() : (nil)
State                  : EGL_NO_DISPLAY
```

State Karşılaştırması

Durum	eglGetCurrentDisplay()
Henüz current context yok	EGL_NO_DISPLAY
Display initialized, context current değil	EGL_NO_DISPLAY
Context current	İlgili EGLDisplay
Current context release edildi	EGL_NO_DISPLAY

eglGetDisplay ile Farkı

eglGetDisplay :

```
EGLDisplay eglGetDisplay(EGLNativeDisplayType display_id);
```

Native display'den EGL display handle'ı elde eder.

eglGetCurrentDisplay :

```
EGLDisplay eglGetCurrentDisplay(void);
```

Current context'in bağlı olduğu display'i sorgular.

```
eglGetDisplay
native display -> EGLDisplay

eglGetCurrentDisplay
current context -> ilişkili EGLDisplay
```

eglInitialize ile Farkı

```
eglInitialize
|
v
Display EGL kullanımı için initialize edilir
```

Bu tek başına current state oluşturmaz.

Deneyde:

```
eglInitialize başarılı
|
v
eglGetCurrentDisplay == EGL_NO_DISPLAY
```

gözlemlendi.

`eglMakeCurrent` ile İlişki

`eglMakeCurrent` , current state'i değiştirir:

```
eglMakeCurrent(dpy, surface, surface, ctx);
```

Başarılı çağrıdan sonra `eglGetCurrentDisplay()` ilgili display'i döndürdü.

Release:

```
eglMakeCurrent(
    dpy,
    EGL_NO_SURFACE,
    EGL_NO_SURFACE,
    EGL_NO_CONTEXT
);
```

sonrasında tekrar `EGL_NO_DISPLAY` döndü.

`eglMakeCurrent` ayrıntıları için ilgili bölüme bakınız.

`EGL_NO_DISPLAY` Her Zaman Hata Değildir

`eglGetCurrentDisplay()` için:

```
EGL_NO_DISPLAY
```

current context bulunmadığında normal bir state sonucudur.

Örneğin:

```
Program yeni başladı
Current context yok
eglGetCurrentDisplay -> EGL_NO_DISPLAY
```

Bu tek başına “EGL API hatası oluştu” anlamına gelmez.

Thread Notu

Current state thread-local'dır.

`eglGetCurrentDisplay` :

“Programın herhangi bir yerinde kullanılan display hangisi?”

sorusunu değil:

“Bu çağrıyı yapan thread'in current context'i hangi display'e bağlı?”

sorusunu cevaplar.

Bu çalışmadaki testler tek thread üzerinde yapıldı.

Minimal Kullanım

```
EGLDisplay current = eglGetCurrentDisplay();

if (current == EGL_NO_DISPLAY) {
    printf("Current EGL display yok.\n");
} else {
    printf("Current EGLDisplay: %p\n", (void *)current);
}
```

EGL 1.0 İçin Pratik Özet

- `eglGetCurrentDisplay` parametre almaz.
- Dönüş tipi `EGLDisplay` 'dir.
- Current context varsa onun bağlı olduğu display'i döndürür.
- Current context yoksa `EGL_NO_DISPLAY` döndürür.
- Display'in yalnızca oluşturulmuş veya initialize edilmiş olması yeterli değildir.
- Current state `eglMakeCurrent` ile oluşturulur/değiştirilir.
- Current context release edilince sonuç tekrar `EGL_NO_DISPLAY` olur.
- `EGL_NO_DISPLAY` bu fonksiyonda normal bir state sonucu olabilir.
- Current state thread-local'dır.

EGL 1.0: eglGetCurrentContext

```
EGLContext eglGetCurrentContext(void);
```

`eglGetCurrentContext` , çağırıcı yapan thread (iş parçacığı) için o anda aktif olan (state machine'e bağlanmış) `EGLContext` nesnesinin handle (tanıtıcı) değerini döndürür. Bu fonksiyon, herhangi bir argüman almadan doğrudan thread-local bellek üzerinden durumu okuduğu için sistem durumunu sorgulamada en düşük maliyetli EGL fonksiyonlarından biridir.

Mental Model

```
İşletim Sistemi (OS) - Thread Local Storage (TLS)
├─ İş Parçacığı (Thread ID: 0x1234)
│   └─ Aktif EGL State Makinesi
│       └─ EGLContext (Current Context - eglGetCurrentContext(
│           └─ EGLDisplay (Örn: DRM/GBM Display veya X11 Disp
│               └─ EGLSurface (DRAW - Çizim Hedefi)
│                   └─ NativeWindowType (Örn: gbm_surface veya W
│                       └─ EGLSurface (READ - Okuma Hedefi)
│                           └─ NativePixmapType / NativeWindowType
└─ [Alternatif] EGL_NO_CONTEXT (Hiçbir context bağlanmamış)
```

Parametreler

Bu fonksiyon spesifikasyona göre hiçbir parametre almaz (`void`).

void

Değer	Sonuç
<code>void</code> (Parametre yok)	Thread'e bağlı geçerli bir <code>EGLContext</code> varsa o döndürülür, aksi takdirde <code>EGL_NO_CONTEXT</code> döner.

Not: Fonksiyon parametre almadığı için `NativeWindowType` veya `NativePixmapType` gibi platforma bağımlı tipleri doğrudan etkilemez veya kabul etmez. Ancak dönen context'in dolaylı olarak bağlı olduğu yüzeyler (surface) X11'de Window, DRM/GBM sistemlerinde ise `gbm_surface` gibi yapılara tekabül eder.

Geçerli Attribute Listesi ve Tampon (Buffer) Tipleri

`eglGetCurrentContext` fonksiyonu konfigürasyon veya flag dizisi almaz. Ancak dönen context'in durumunu değerlendirirken tampon yapıları büyük

önem taşır:

Attribute / Durum	Tip	Anlam
Konfigürasyon Listesi	Yok	Fonksiyon attribute dizisi kabul etmez.
Back Buffered (Çift Tamponlu)	Surface Durumu	Dönen context bir Window yüzeyine bağlıysa, çizimler arka tampona (back buffer) yapılır ve <code>eglSwapBuffers</code> beklenir.
Single Buffered (Tek Tamponlu)	Surface Durumu	Dönen context bir Pixmap veya Pbuffer (off-screen) yüzeyine bağlıysa, çizimler anında gerçekleştirilebilir ve frame eşzamanlaması farklı yönetilir.

Ayrıntılar ve Yaşam Döngüsü

Thread Güvenliği (Thread Safety): EGL standartlarına göre thread güvenliği son derece katıdır. Bir `EGLContext` aynı anda sadece **bir** thread üzerinde current (aktif) olabilir. Eğer aynı context başka bir thread'de zaten aktifse ve o thread'de `eglMakeCurrent` çağrılmazsa, farklı bir thread üzerinden bu context'e erişilemez. `eglGetCurrentContext` fonksiyonu thread-safe'tir ve yalnızca çağıran thread'in belleğine bakar, diğer thread'leri bloklamaz (non-blocking).

Eşzamanlama (Synchronization): Dönen context üzerinden çizim komutları gönderilirken, OpenGL ile Native API (Örn: DRM veya X11 2D render komutları) arasında eşzamanlama gerekiyorsa `eglWaitGL` (OpenGL komutlarının bitmesini bekler) ve `eglWaitNative` (Native render komutlarının bitmesini bekler) kullanılmalıdır. Native objeler (Window/Pixmap) üzerinde OpenGL harici bir API işlem yapacaksa, context üzerinde `glFinish()` çağırmak da alternatif bir donanım bazlı eşzamanlama garantisidir.

Hata Matrisi

Khronos spesifikasyonlarına göre `eglGetCurrentContext`, state machine üzerinde hiçbir değişiklik yapmayan **yan etkisiz (side-effect free)** bir okuma (getter) operasyonudur. Başarısızlık durumunda dahi EGL error flag'lerini modifiye etmez.

Durum	Sonuç (EGL Hata Kodu)
-------	-----------------------

Durum	Sonuç (EGL Hata Kodu)
EGL hiç initialize edilmediyse	<code>EGL_NO_CONTEXT</code> döner, <code>eglGetError()</code> değişmez.
Başka thread'de context aktif, mevcut thread'de boşsa	<code>EGL_NO_CONTEXT</code> döner, <code>eglGetError()</code> değişmez.
<code>eglMakeCurrent</code> başarıyla çalıştıysa	Aktif <code>EGLContext</code> handle'ı döner, hata kodu değişmez.

*Önemli Not: Bu fonksiyon hiçbir zaman `EGL_BAD_MATCH` ,
`EGL_BAD_NATIVE_WINDOW` gibi hatalar fırlatmaz. Bir şeylerin ters gittiğini sadece
dönen değerin `EGL_NO_CONTEXT` olmasından anlayabilirsiniz.*

Güvenli Kullanım Örneği

Aşağıdaki örnekte, fonksiyonun DRM/GBM veya gömülü sistemler fark
etmeksizin nasıl güvenli kullanılacağı gösterilmiştir:


```
#include <EGL/egl.h>
#include <stdio.h>

// Context'in dogru sekilde current (aktif) olup olmadigini test et
void perform_safe_rendering(EGLDisplay dpy, EGLSurface draw, EGLSu:
    // 1. Context'i aktif yapmayi dene
    if (!eglMakeCurrent(dpy, draw, read, ctx)) {
        EGLint err = eglGetError();
        fprintf(stderr, "Kritik Hata: eglMakeCurrent basarisiz oldu
        return;
    }

    // 2. State machine'in gercekten context'i kabul ettigini dogru
    EGLContext current_ctx = eglGetCurrentContext();

    if (current_ctx == EGL_NO_CONTEXT) {
        // eglMakeCurrent basarili donse bile spesifikasyon disi de
        fprintf(stderr, "Fatal: eglMakeCurrent basarili oldu fakat
        // Guvenli cikis (Memory leak ve undefined behavior onleme
        eglMakeCurrent(dpy, EGL_NO_SURFACE, EGL_NO_SURFACE, EGL_NO
        return;
    }

    if (current_ctx != ctx) {
        fprintf(stderr, "Fatal: Aktif context (%p), beklenen conte
            (void*)current_ctx, (void*)ctx);
        eglMakeCurrent(dpy, EGL_NO_SURFACE, EGL_NO_SURFACE, EGL_NO
        return;
    }

    // 3. Guvenli bolge: Context kesin olarak bu thread'e bagli
    printf("Context basariyla dogrulandi (%p). Cizim islemleri bas:

    // glClear(GL_COLOR_BUFFER_BIT) vs...

    // 4. Eşzamanlama (Opsiyonel ama güvenlik kritik sistemlerde ön
    // eglWaitGL();

    // 5. Cizim bittikten sonra context'i guvenli sekilde ayir (Cle
    eglMakeCurrent(dpy, EGL_NO_SURFACE, EGL_NO_SURFACE, EGL_NO_CON
}
```

Pratik Özet

- **Maliyet ve Yan Etki:** Çok ucuz (düşük maliyetli) ve tamamen yan etkisiz (side-effect free) bir çağrıdır. `eglGetError()` state'ini asla kirletmez.
- **Thread Local Davranış:** Sonuç tamamen thread-spesifiktir. Her thread sadece kendi aktif context'ini görebilir.
- **Doğrulama (Validation):** `eglMakeCurrent` sonrası sistemin (özellikle gömülü GPU sürücülerinin) gerçekten state değişikliğini yansıtıp yansıtmadığını çift kontrol (double-check) etmek için kritik bir diagnostik araçtır.
- **Platform Bağımsızlığı:** Dönen değer `EGLContext` olup X11, Wayland, QNX veya DRM/GBM sistemlerindeki pencerelendirme mimarilerinden bağımsız, sadece EGL ve OpenGL ES'in dahili durumunu temsil eder.

EGL 1.0: `eglSwapBuffers`

```
EGLBoolean eglSwapBuffers(EGLDisplay dpy,  
                           EGLSurface surface);
```

`eglSwapBuffers` , EGL 1.0'da rendering sonrası color buffer'ı native window'a post etmek için kullanılır.

Kısa özet:

- Window surface için: color buffer native window'a post edilir.
- Pbuffer surface için: etkisi yoktur.
- Pixmap surface için: etkisi yoktur.
- EGL 1.0'da `surface` , çağıran thread'in current context'ine bağlı olmalıdır.

Mental Model

Window surface için:

```
OpenGL ES draw commands
    |
    v
EGL window surface back/color buffer
    |
    v
eglSwapBuffers
    |
    v
native window'a post/copy
```

GBM + DRM/KMS için daha uzun zincir gerekir:

```
OpenGL ES
    |
    v
EGLSurface
    |
    v
GBM surface
    |
    v
eglSwapBuffers
    |
    v
GBM front BO hazır olur
    |
    v
gbm_surface_lock_front_buffer
    |
    v
DRM framebuffer
    |
    v
drmModeSetCrtc / drmModePageFlip
    |
    v
monitor
```

Parametreler

dpy

Değer	Sonuç
Geçerli ve initialized EGLDisplay	Diğer parametreler de geçerliyse çağrı çalışır.
EGL_NO_DISPLAY	Başarısız. Genel EGL hata modeliyle EGL_BAD_DISPLAY beklenir.
Geçersiz display handle	Başarısız. Genel EGL hata modeliyle EGL_BAD_DISPLAY beklenir.
Initialize edilmemiş display	Başarısız. Genel EGL hata modeliyle EGL_NOT_INITIALIZED beklenir.

surface

Değer	Sonuç
Current context'e bağlı geçerli window surface	Color buffer native window'a post edilir.
Current context'e bağlı geçerli pbuffer surface	EGL 1.0'a göre etkisi yoktur.
Current context'e bağlı geçerli pixmap surface	EGL 1.0'a göre etkisi yoktur.
Geçerli ama current context'e bağlı olmayan surface	Başarısız, EGL_BAD_SURFACE .
EGL_NO_SURFACE	Başarısız, EGL_BAD_SURFACE .
Geçersiz surface	Başarısız, EGL_BAD_SURFACE .
Yok edilmiş surface	Başarısız, EGL_BAD_SURFACE veya tanımsız native durum.
Native window'u geçersiz window surface	Başarısız, EGL_BAD_NATIVE_WINDOW .

EGL 1.0 Current Surface Şartı

EGL 1.0 spec'i için önemli kural:

surface, çağıran thread'in current context'ine bağlı olmalıdır.

Doğru sıra:

```
eglMakeCurrent(dpy, surface, surface, ctx);

/* OpenGL ES çizimleri */

eglSwapBuffers(dpy, surface);
```

Yanlış sıra:

```
/* surface current yapılmadı */
eglSwapBuffers(dpy, surface);
```

Beklenen hata:

```
EGL_FALSE
EGL_BAD_SURFACE
```

Surface Tiplerine Göre Davranış

Window Surface

```
eglSwapBuffers(dpy, window_surface);
```

Başarı için:

- `dpy` initialized olmalı.
- `window_surface` geçerli olmalı.
- `window_surface` current context'e bağlı olmalı.
- Native window hala geçerli olmalı.

Başarı sonucu:

- `EGL_TRUE` döner.
- Color buffer native window'a post edilir.
- Surface color buffer içeriği swap sonrası tanımsız olur.
- Current context için implicit `glFlush` yapılır.

EGL 1.0 metni "color buffer copied to the native window" şeklinde tarif eder. Gerçek implementation bunu copy, page flip, buffer exchange veya platform-specific başka bir yolla gerçekleştirebilir; uygulama EGL 1.0 seviyesinde bu mekanizmaya güvenmemelidir.

Pbuffer Surface

```
eglSwapBuffers(dpy, pbuffer_surface);
```

EGL 1.0'a göre etkisi yoktur.

Pbuffer offscreen surface'tir:

```
pbuffer -> native visible window yok
```

Bu yüzden `eglSwapBuffers` fiziksel görüntü üretmez. Pbuffer içeriğini kullanmak istiyorsan tipik yollar:

- aynı context içinde texture/copy/readback akışı
- `glReadPixels`
- başka EGL/OpenGL ES mekanizmaları

EGL 1.0 açısından `eglSwapBuffers` pbuffer için görüntü post etmez.

Pixmap Surface

```
eglSwapBuffers(dpy, pixmap_surface);
```

EGL 1.0'a göre etkisi yoktur.

Pixmap'e copy yapmak için ilişkili fonksiyon `eglCopyBuffers` 'tır:

```
eglCopyBuffers(dpy, surface, native_pixmap);
```

`eglSwapBuffers` , pixmap hedefli copy fonksiyonu değildir.

Hata Matrisi

Durum	Sonuç
Window surface current ve native window geçerli	<code>EGL_TRUE</code> ; post yapılır.
Pbuffer surface current	<code>EGL_TRUE</code> dönebilir; etkisi yoktur.
Pixmap surface current	<code>EGL_TRUE</code> dönebilir; etkisi yoktur.
Surface current değil	<code>EGL_FALSE</code> , <code>EGL_BAD_SURFACE</code> .
Surface geçersiz	<code>EGL_FALSE</code> , <code>EGL_BAD_SURFACE</code> .

Durum	Sonuç
<code>surface == EGL_NO_SURFACE</code>	<code>EGL_FALSE</code> , <code>EGL_BAD_SURFACE</code> .
Native window artık geçersiz	<code>EGL_FALSE</code> , <code>EGL_BAD_NATIVE_WINDOW</code> .
<code>dpy</code> geçersiz	<code>EGL_FALSE</code> , tipik hata <code>EGL_BAD_DISPLAY</code> .
<code>dpy</code> initialized değil	<code>EGL_FALSE</code> , tipik hata <code>EGL_NOT_INITIALIZED</code> .

Swap Sonrası Color Buffer Neden Tanımsız?

EGL 1.0 spec'i başarılı `eglSwapBuffers` sonrası surface color buffer içeriğinin tanımsız olduğunu söyler.

Yani şu varsayım yanlıştır:

```
eglSwapBuffers(dpy, surface);

/* Yanlış varsayım: eski color buffer içeriği hala korunuyor */
```

Doğru pratik:

```
/* Her frame'de gerekli içeriği yeniden çiz */
glClear(...);
draw_scene();
eglSwapBuffers(dpy, surface);
```

EGL 1.0 seviyesinde buffer preservation garantisi yoktur.

Implicit `glFlush`

EGL 1.0 posting semantics:

Eğer `dpy` ve `surface` çağırılan thread'in current context'ine aitse, `eglSwapBuffers` implicit `glFlush` yapar.

Bu `glFinish` değildir.

Fonksiyon	Anlam
-----------	-------

Fonksiyon	Anlam
<code>glFlush</code>	Komutların GPU'ya gönderilmesini başlatır; tamamlanmasını beklemek zorunda değildir.
<code>glFinish</code>	Önceki GL komutlarının tamamlanmasını bekler.
<code>eglSwapBuffers</code>	Posting sırasında implicit <code>glFlush</code> yapar; swap/post tamamlanması implementation'a bağlıdır.

EGL 1.0 metni, sonraki OpenGL ES komutlarının hemen verilebileceğini ama posting bitene kadar yürütülmeyebileceğini belirtir. Window surface için bu zamanlama tipik olarak vertical retrace ile ilişkilidir.

Native Window Resize

Eğer native window swap öncesinde resize edilmişse, EGL surface native window ile uyumlu hale gelmelidir.

EGL implementation resize'ı daha önce şeffaf şekilde yapamadıysa, `eglSwapBuffers` bunu fark edip surface'i post öncesinde resize etmelidir.

Sonuçlar:

- Surface küçülürse bazı çizilmiş pixel'ler kaybolur.
- Surface büyürse yeni buffer alanlarının içeriği tanımsızdır.
- Uygulama kendi viewport/scissor ayarlarını güncellemekten sorumludur.

Örnek:

```
/* Native window 800x600 -> 1024x768 olduysa */
glViewport(0, 0, 1024, 768);
draw_scene();
eglSwapBuffers(dpy, surface);
```

EGL surface resize edebilir, ama GL viewport'u senin uygulamanın sorumluluğundadır.

`eglSwapBuffers` ve DRM/KMS

EGL tek başına fiziksel monitörü yönetmez. Bu özellikle GBM + DRM/KMS tarafında önemlidir.

`eglSwapBuffers` sonrası:

```
GBM surface içinde render edilmiş front buffer hazır olur.
```


Ama fiziksel monitör için hala şu gerekir:

```
struct gbm_bo *bo = gbm_surface_lock_front_buffer(gbm_surface);

/* bo handle/stride/format -> DRM framebuffer */
drmModeAddFB2(fd, width, height, format, handles, strides, offsets

/* CRTC artık bu framebuffer'ı scanout etsin */
drmModeSetCrtc(fd, crtc_id, fb_id, 0, 0, &connector_id, 1, &mode);
```

Animasyonda sonraki buffer'a geçmek için:

```
drmModePageFlip(fd, crtc_id, next_fb_id, DRM_MODE_PAGE_FLIP_EVENT,
```

Özet:

Katman	Sorumluluk
OpenGL ES	Pixel üretir.
EGL	Context/surface bağlar ve swap/post işlemini yapar.
GBM	GPU/display paylaşılabılır buffer nesneleri sağlar.
DRM/KMS	Framebuffer'ı CRTC/connector üzerinden monitöre scanout eder.

Minimal Window Surface Kullanımı

```
eglMakeCurrent(dpy, window_surface, window_surface, ctx);

glViewport(0, 0, width, height);
glClearColor(0.0f, 0.2f, 0.8f, 1.0f);
glClear(GL_COLOR_BUFFER_BIT);

if (!eglSwapBuffers(dpy, window_surface)) {
    EGLint err = eglGetError();
}
```

Minimal Pbuffer Kullanımı

```
eglMakeCurrent(dpy, pbuffer_surface, pbuffer_surface, ctx);

glClearColor(1.0f, 0.0f, 0.0f, 1.0f);
glClear(GL_COLOR_BUFFER_BIT);

eglSwapBuffers(dpy, pbuffer_surface); /* EGL 1.0: no effect */
```

Pbuffer için `eglSwapBuffers` çağrısı öğretici olabilir ama görünür output beklenmemelidir.

Bilerek Hata Üretme Örnekleri

```
/* EGL_BAD_SURFACE: surface yok */
eglSwapBuffers(dpy, EGL_NO_SURFACE);

/* EGL_BAD_SURFACE: surface current değil */
eglMakeCurrent(dpy, EGL_NO_SURFACE, EGL_NO_SURFACE, EGL_NO_CONTEXT)
eglSwapBuffers(dpy, surface);

/* EGL_BAD_NATIVE_WINDOW: native window yok edilmişse */
destroy_native_window();
eglSwapBuffers(dpy, window_surface);
```

EGL 1.0 İçin Pratik Özet

- `eglSwapBuffers` window surface için anlamlıdır.
- Pbuffer ve pixmap surface için etkisi yoktur.
- EGL 1.0'da surface current context'e bağlı olmalıdır.
- Başarılı swap sonrası color buffer içeriğini korunmuş sayma.
- `eglSwapBuffers` implicit `glFlush` yapar ama `glFinish` değildir.
- GBM/DRM kullanıyorsan swap sonrası ayrıca BO alma ve KMS scanout gerekir.

EGL 1.0: `eglDestroyContext`

```
EGLBoolean eglDestroyContext(EGLDisplay dpy,
                             EGLContext ctx);
```

`eglDestroyContext` , daha önce oluşturulmuş bir `EGLContext` nesnesini yok edilmek üzere işaretler.

EGL 1.0'a göre context o anda herhangi bir thread üzerinde current değilse kaynakları mümkün olan en kısa sürede serbest bırakılır. Context current durumdaysa `eglDestroyContext` çağrısından sonra hemen ortadan kalkmaz; current kaldığı sürece kullanılmaya devam eder ve ilgili thread üzerinde sonraki geçerli `eglMakeCurrent` çağrısında gerçek anlamda yok edilir.

Mental Model

Normal context yaşam döngüsü:

```
eglCreateContext()  
  |  
  v  
EGLContext  
  |  
  v  
eglMakeCurrent()  
  |  
  v  
OpenGL ES rendering  
  |  
  v  
eglMakeCurrent(... EGL_NO_CONTEXT)  
  |  
  v  
eglDestroyContext()  
  |  
  v  
Context kaynakları serbest bırakılır
```

Context current iken destroy edilirse:

```

EGLContext current
    |
    v
eglDestroyContext()
    |
    v
"silinmek üzere işaretli"
    |
    | current olduğu sürece yaşamaya devam eder
    v
sonraki geçerli eglMakeCurrent()
    |
    v
gerçek kaynak serbest bırakma

```

Parametreler

dpy

dpy , yok edilecek context'in ait olduğu `EGLDisplay` nesnesidir.

Değer	Sonuç
Context'in oluşturulduğu geçerli <code>EGLDisplay</code>	Normal kullanım.
GBM tabanlı initialized <code>EGLDisplay</code>	Bu projede kullanılacak display türüdür.

Bu projede:

```

/dev/dri/card*
    |
    v
gbm_device
    |
    v
EGLDisplay
    |
    +-- EGLContext

```

Örnek:

```
EGLBoolean ok = eglDestroyContext(  
    egl_display,  
    egl_context  
);
```

Pratik olarak:

```
dpy = egl_context'in ait olduğu initialized EGLDisplay
```

olmalıdır.

ctx

ctx , yok edilmek üzere işaretlenecek rendering context handle'ıdır.

Normal olarak daha önce:

```
EGLContext egl_context = eglCreateContext(  
    egl_display,  
    egl_config,  
    EGL_NO_CONTEXT,  
    NULL  
);
```

ile oluşturulmuş bir context verilir.

Ardından:

```
eglDestroyContext(  
    egl_display,  
    egl_context  
);
```

çağrılır.

Context current değilse

```
eglMakeCurrent(  
    egl_display,  
    EGL_NO_SURFACE,  
    EGL_NO_SURFACE,  
    EGL_NO_CONTEXT  
);  
  
eglDestroyContext(  
    egl_display,  
    egl_context  
);
```

Bu en temiz kapanış biçimidir.

Context current ise

EGL 1.0'a göre:

```
eglDestroyContext(  
    egl_display,  
    egl_context  
);
```

çağrısı context'i hemen kullanılamaz hale getirmez.

Context:

```
current  
+  
destroy çağrılmış
```

durumunda ise kaynakları current kaldığı sürece tutulur.

Thread üzerinde daha sonra geçerli bir:

```
eglMakeCurrent(...)
```

çağrısı yapıldığında eski context artık current olmaktan çıkar ve silme işlemi tamamlanabilir.

Dönüş Değeri

Fonksiyonun dönüş tipi:

EGLBoolean

Başarılı çağrı:

EGL_TRUE

Başarısız çağrı:

EGL_FALSE

Örnek:

```
if (!eglDestroyContext(egl_display, egl_context)) {  
    EGLint err = eglGetError();  
}
```

Kullanım Senaryoları

1. Normal context kapatma

```
eglMakeCurrent(  
    egl_display,  
    EGL_NO_SURFACE,  
    EGL_NO_SURFACE,  
    EGL_NO_CONTEXT  
);  
  
EGLBoolean result =  
    eglDestroyContext(  
        egl_display,  
        egl_context  
    );
```

Sonuç:

Context current değildir.
Context yok edilmek üzere işaretlenir.
Kaynaklar serbest bırakılabilir.

2. Context current iken destroy

```
eglMakeCurrent(  
    egl_display,  
    egl_surface,  
    egl_surface,  
    egl_context  
);  
  
eglDestroyContext(  
    egl_display,  
    egl_context  
);
```

Bu noktada:

```
egl_context hâlâ current
```

olduğu için EGL 1.0 context kaynaklarını hemen bırakmaz.

Sonraki:

```
eglMakeCurrent(  
    egl_display,  
    EGL_NO_SURFACE,  
    EGL_NO_SURFACE,  
    EGL_NO_CONTEXT  
);
```

çağırısından sonra context artık current değildir ve yok edilmesi tamamlanabilir.

3. Direct-to-display proje kapanışı

Bu projede EGL tarafının güvenli kapanış sırası:


```
eglMakeCurrent(  
    egl_display,  
    EGL_NO_SURFACE,  
    EGL_NO_SURFACE,  
    EGL_NO_CONTEXT  
);  
  
eglDestroyContext(  
    egl_display,  
    egl_context  
);  
  
eglDestroySurface(  
    egl_display,  
    egl_surface  
);  
  
eglTerminate(  
    egl_display  
);
```

GBM ve DRM/KMS kaynakları bundan ayrı olarak daha sonra temizlenir.

Örneğin:

```
EGL context  
  ↓  
EGL surface  
  ↓  
EGL terminate  
  ↓  
GBM surface/device  
  ↓  
DRM resources/fd
```

Parametre Matrisi

dpy	ctx	Sonuç
Geçerli EGLDisplay	Current olmayan geçerli EGLContext	Context yok edilir.

dpy	ctx	Sonuç
Geçerli EGLDisplay	Current olan geçerli EGLContext	Silinmek üzere işaretlenir; current kaldığı sürece kaynakları tutulur.
Geçerli EGLDisplay	Geçersiz context	Çağrı başarısız olur.

EGL 1.0 Hata Kodu

EGL 1.0 bu fonksiyon için doğrudan şu hatayı tanımlar:

Hata	Ne zaman
EGL_BAD_CONTEXT	ctx geçerli bir EGL rendering context değilse.

Fonksiyon başarısız olduğunda:

```
EGLint err = eglGetError();
```

ile hata kodu okunabilir.

GBM / DRM ile İlişkisi

eglDestroyContext yalnızca EGL context nesnesini yönetir.

Şunları yok etmez:

```
gbm_surface
gbm_device
GBM BO
DRM framebuffer
DRM connector
DRM CRTC
DRM file descriptor
```

Yani:

```
eglDestroyContext()  
|  
+-- EGLContext temizliği  
|  
X-- GBM surface temizliği değil  
X-- DRM/KMS temizliği değil
```

GBM için ayrıca:

```
gbm_surface_destroy(gbm_surface);  
gbm_device_destroy(gbm);
```

gibi ilgili GBM fonksiyonları kullanılır.

Minimal Doğru Kullanım

```
/* Context'i thread'den ayır */  
if (!eglMakeCurrent(  
    egl_display,  
    EGL_NO_SURFACE,  
    EGL_NO_SURFACE,  
    EGL_NO_CONTEXT)) {  
    EGLint err = eglGetError();  
}  
  
/* Context'i yok et */  
if (!eglDestroyContext(  
    egl_display,  
    egl_context)) {  
    EGLint err = eglGetError();  
}
```

Parametre Değiştirme Örnekleri

1. Current olmayan context

```
eglMakeCurrent(  
    egl_display,  
    EGL_NO_SURFACE,  
    EGL_NO_SURFACE,  
    EGL_NO_CONTEXT  
);  
  
eglDestroyContext(  
    egl_display,  
    egl_context  
);
```

ctx = geçerli ve current olmayan context
Sonuç = kaynaklar serbest bırakılabilir

2. Current context

```
eglDestroyContext(  
    egl_display,  
    egl_context  
);
```

ctx = current context
Sonuç = context silinmek üzere işaretlenir,
fakat current kaldığı sürece yaşamaya devam eder

3. Farklı context'ler

```
eglDestroyContext(egl_display, context_a);  
eglDestroyContext(egl_display, context_b);
```

context_a ve context_b bağımsız EGLContext nesneleridir.
Hangi handle verilirse o context yok edilmek üzere işaretlenir.

EGL 1.0 İçin Pratik Özet

- `eglDestroyContext` , bir `EGLContext` nesnesini yok edilmek üzere işaretler.
- `dpy` , context'in ait olduğu `EGLDisplay` nesnesidir.

- `ctx` , yok edilecek rendering context'tir.
- Başarılı çağrı `EGL_TRUE` , başarısız çağrı `EGL_FALSE` döndürür.
- `ctx` current değilse kaynakları serbest bırakılabilir.
- `ctx` current ise hemen yok edilmez.
- Current context'in gerçek silinmesi, thread üzerinde sonraki geçerli `eglMakeCurrent` çağrısıyla tamamlanır.
- Bu fonksiyon GBM veya DRM/KMS kaynaklarını yok etmez.

EGL 1.0: `eglDestroySurface`

```
EGLBoolean eglDestroySurface(EGLDisplay dpy,  
                             EGLSurface surface);
```

`eglDestroySurface` , daha önce oluşturulmuş bir `EGLSurface` nesnesini yok edilmek üzere işaretler.

Fonksiyon window, pbuffer veya pixmap türündeki EGL surface'lerde kullanılabilir. Bu projede ise `eglCreateWindowSurface` ile GBM native surface üzerinde oluşturulan window `EGLSurface` yok edilir.

Mental Model

Bu projedeki surface zinciri:

```
struct gbm_surface *  
    |  
    v  
eglCreateWindowSurface()  
    |  
    v  
EGLSurface  
    |  
    v  
eglMakeCurrent()  
    |  
    v  
OpenGL ES rendering  
    |  
    v  
eglDestroySurface()
```

Önemli ayrım:

```
eglDestroySurface() -> EGLSurface'i yönetir  
gbm_surface_destroy() -> struct gbm_surface * nesnesini yönetir
```

Bunlar aynı nesne değildir.

Parametreler

dpy

`dpy` , yok edilecek surface'in ait olduğu `EGLDisplay` nesnesidir.

Bu projede:

```
GBM native platform  
  |  
  v  
EGLDisplay  
  |  
+-- EGLSurface
```

Örnek:

```
eglDestroySurface(  
    egl_display,  
    egl_surface  
);
```

Pratik kullanım:

```
dpy = egl_surface'in oluşturulduğu initialized EGLDisplay
```

surface

`surface` , yok edilmek üzere işaretlenecek `EGLSurface` handle'ıdır.

Bu projede daha önce:

```
EGLSurface egl_surface =  
    eglCreateWindowSurface(  
        egl_display,  
        egl_config,  
        (EGLNativeWindowType)gbm_surface,  
        NULL  
    );
```

ile oluşturulmuştur.

Sonra:

```
eglDestroySurface(  
    egl_display,  
    egl_surface  
);
```

çağrılır.

Surface current değilse

Surface hiçbir thread'de current draw/read surface değilse, kaynakları mümkün olan en kısa sürede bırakılabilir.

```
EGLSurface  
|  
| current değil  
v  
eglDestroySurface()  
|  
v  
kaynaklar serbest bırakılır
```

Surface current ise

EGL 1.0'a göre current bir surface destroy edildiğinde kaynakları hemen serbest bırakılmaz.

```
surface current
|
v
eglDestroySurface()
|
v
silinmek üzere işaretlenir
|
| current kaldığı sürece yaşamaya devam eder
v
sonraki geçerli eglMakeCurrent()
|
v
gerçek yok etme tamamlanabilir
```

Bu nedenle güvenli kullanımda önce context/surface bağı kaldırılır.

Dönüş Değeri

Fonksiyon:

EGLBoolean

döndürür.

Başarı:

EGL_TRUE

Başarısızlık:

EGL_FALSE

Örnek:

```
if (!eglDestroySurface(
    egl_display,
    egl_surface)) {
    EGLint err = eglGetError();
}
```

Kullanım Senaryoları

1. Normal surface kapatma

Önce current bağ kaldırılır:

```
eglMakeCurrent(  
    egl_display,  
    EGL_NO_SURFACE,  
    EGL_NO_SURFACE,  
    EGL_NO_CONTEXT  
);
```

Ardından:

```
eglDestroySurface(  
    egl_display,  
    egl_surface  
);
```

Bu proje için en temiz kullanım şeklidir.

2. Current surface üzerinde destroy

Örneğin:

```
eglMakeCurrent(  
    egl_display,  
    egl_surface,  
    egl_surface,  
    egl_context  
);  
  
eglDestroySurface(  
    egl_display,  
    egl_surface  
);
```

EGL 1.0'a göre surface current kaldığı için kaynakları hemen bırakılmaz.

Daha sonra:

```
eglMakeCurrent(  
    egl_display,  
    EGL_NO_SURFACE,  
    EGL_NO_SURFACE,  
    EGL_NO_CONTEXT  
);
```

çağrısıyla current bağı değiştiğinde surface'in yok edilmesi tamamlanabilir.

3. GBM native surface ile ilişki

Bu projede:

```
gbm_surface  
  |  
  v  
EGLSurface
```

ilişkisi vardır.

Ancak:

```
eglDestroySurface(  
    egl_display,  
    egl_surface  
);
```

yalnızca EGL katmanını temizler.

Native GBM surface ayrıca:

```
gbm_surface_destroy(gbm_surface);
```

ile temizlenmelidir.

Doğru mantık:

```

EGLSurface'i bırak
    |
    v
eglDestroySurface()
    |
    v
GBM surface artık EGL tarafından kullanılmıyor
    |
    v
gbm_surface_destroy()

```

Parametre Matrisi

dpy	surface	Sonuç
Geçerli EGLDisplay	Current olmayan geçerli EGLSurface	Surface yok edilir.
Geçerli EGLDisplay	Current geçerli EGLSurface	Silinmek üzere işaretlenir; current kaldığı sürece kaynakları tutulur.
Geçerli EGLDisplay	Geçersiz EGLSurface	Başarısız.

EGL 1.0 Hata Kodu

EGL 1.0 bu fonksiyon için doğrudan:

Hata	Ne zaman
EGL_BAD_SURFACE	surface geçerli bir EGL rendering surface değilse.

tanımlar.

Başarısız çağrı:

```
EGL_FALSE
```

döndürür.

Ardından:

```
EGLint err = eglGetError();
```

ile hata alınabilir.

Direct-to-Display Projede Kapanış

Örneğin:

```
eglMakeCurrent(  
    egl_display,  
    EGL_NO_SURFACE,  
    EGL_NO_SURFACE,  
    EGL_NO_CONTEXT  
);  
  
eglDestroyContext(  
    egl_display,  
    egl_context  
);  
  
eglDestroySurface(  
    egl_display,  
    egl_surface  
);  
  
eglTerminate(  
    egl_display  
);  
  
gbm_surface_destroy(  
    gbm_surface  
);  
  
gbm_device_destroy(  
    gbm  
);
```

DRM/KMS tarafındaki framebuffer, connector/CRTC state ve file descriptor da kendi API'leriyle ayrıca temizlenir.

GBM BO Notu

Direct-to-display akışında render sonrasında:

```
struct gbm_bo *bo =  
    gbm_surface_lock_front_buffer(gbm_surface);
```

ile front buffer kilitlenmiş olabilir.

Bu buffer'ın yaşam döngüsü `EGLSurface` yaşam döngüsünden farklıdır.

Dolayısıyla cleanup sırasında:

```
EGLSurface  
GBM surface  
GBM BO  
DRM framebuffer
```

nesnelerinin aynı şey olmadığı unutulmamalıdır.

`eglDestroySurface()` yalnızca ilkinin yönetir.

Minimal Doğru Kullanım

```
/* EGL surface/context başını kaldır */  
if (!eglMakeCurrent(  
    egl_display,  
    EGL_NO_SURFACE,  
    EGL_NO_SURFACE,  
    EGL_NO_CONTEXT)) {  
    EGLint err = eglGetError();  
}  
  
/* EGL surface'i yok et */  
if (!eglDestroySurface(  
    egl_display,  
    egl_surface)) {  
    EGLint err = eglGetError();  
}  
  
/* EGL surface artık GBM surface'i kullanmıyor */  
gbm_surface_destroy(gbm_surface);
```

Parametre Değiştirme Örnekleri

1. Current olmayan surface

```
eglDestroySurface(  
    egl_display,  
    egl_surface  
);
```

surface = geçerli ve current olmayan EGLSurface
Sonuç = surface kaynakları serbest bırakılabilir

2. Current surface

```
eglDestroySurface(  
    egl_display,  
    egl_surface  
);
```

surface = current EGLSurface
Sonuç = hemen tamamen yok edilmez;
silinmek üzere işaretlenir

3. Farklı surface'ler

```
eglDestroySurface(  
    egl_display,  
    surface_a  
);  
  
eglDestroySurface(  
    egl_display,  
    surface_b  
);
```

surface_a ve surface_b bağımsız EGLSurface handle'larıdır.
Hangi handle verilirse o surface yok edilmek üzere işaretlenir.

EGL 1.0 İçin Pratik Özet

- `eglDestroySurface` , herhangi bir türdeki `EGLSurface` nesnesini yok edilmek üzere işaretler.
- `dpy` , surface'in ait olduğu `EGLDisplay` nesnesidir.

- `surface` , yok edilecek EGL rendering surface'tir.
- Başarıda `EGL_TRUE` , başarısızlıkta `EGL_FALSE` döner.
- Current olmayan surface mümkün olan en kısa sürede serbest bırakılır.
- Current surface hemen silinmez; current kaldığı sürece kaynakları tutulur.
- Bu projede `eglDestroySurface` GBM surface'i yok etmez.
- `struct gbm_surface *` ayrıca GBM API ile temizlenmelidir.

EGL 1.0: eglTerminate

```
EGLBoolean eglTerminate(EGLDisplay dpy);
```

`eglTerminate` , belirtilen EGL görüntü (display) bağlantısı ile ilişkili tüm EGL kaynaklarını (context, surface vb.) serbest bırakarak donanım/pencere sistemi ile olan EGL oturumunu sonlandırır. EGL API kullanım döngüsünün en son adımıdır ve başlatılmış bir state machine'i bellekten tamamen temizleyerek "uninitialized" (başlatılmamış) durumuna geri döndürür.

Mental Model

```

İşletim Sistemi (OS) & Native Katman                                EGL Soyutlama Katmanı
=====
[Native Display] (X11 Display / DRM fd) <----- EGLDisplay (İ)
|
|
|
+-- [Native Window / GBM Surface] <----- |-- EGLContext
|
|
|
+-- [Native Pixmap / GBM BO] <----- |-- EGLContext

* Current olan nesneler, ilgili thread `eglMakeCurrent(dpy, EGL_NOCON
[ eglTerminate(dpy) Çağrılır ]

EGLDisplay ---> [Uninitialized Duruma Döner]
```

Parametreler

dpy

Değer (Senaryo)	Sonuç
Geçerli ve Initialized Display (Senaryo A)	Display'e bağlı tüm kaynaklar temizlenir veya yıkım için işaretlenir (pending). Display <code>uninitialized</code> durumuna geçer. Fonksiyon <code>EGL_TRUE</code> döndürür.
Geçerli fakat Uninitialized Display (Senaryo C)	Hiçbir işlem yapılmaz. Zaten başlatılmamış bir display'i terminate etmek EGL 1.0 kurallarına göre güvenlidir. Fonksiyon hata üretmez ve <code>EGL_TRUE</code> döndürür.
EGL_NO_DISPLAY veya Geçersiz Pointer (Senaryo B)	Geçersiz bağlantı reddedilir. Fonksiyon hiçbir state değişikliği yapmaz ve <code>EGL_FALSE</code> döndürür. Hata state'i güncellenir.

Native Tip Kısıtlamaları ve Platform Karşılıkları: `EGLDisplay` , EGL'nin native donanım ile kurduğu bağlantıyı temsil eder.

- **DRM/GBM Sistemlerinde (Projemizdeki Altyapı):** Bu parametre doğrudan Linux Kernel DRM file descriptor'unu (`/dev/dri/card0`) ve GBM aygıtını (`gbm_device`) sarmalayan soyut yapıdır. `eglTerminate` çağırıldıktan sonra EGL katmanı kapanır, ancak donanımsal DRM fd ve GBM objeleri EGL'nin sorumluluğunda olmadığından, sonrasında native C fonksiyonları (`gbm_device_destroy` , `close`) ile manuel olarak kapatılmalıdır.
- **X11 / Wayland:** Bu pencere sistemlerinde display pointer'ını temsil eder. `eglTerminate` , `XOpenDisplay` ile açılmış pencere sistemini kapatmaz, sadece EGL wrapper'ını siler.

Geçerli Attribute Listesi

`eglTerminate` fonksiyonu herhangi bir attribute (konfigürasyon) listesi veya flag dizisi **almaz**. Sadece bir adet `EGLDisplay` parametresi ile çalışır.

Yüzey Tiplerine Göre Davranış Farkı: Arka tamponlu (back-buffered) pencere yüzeyleri veya tek tamponlu (single-buffered) pixmap/pbuffer yüzeyleri fark etmeksizin; `eglTerminate` çağırıldığında bu yüzeyler herhangi bir thread'de **aktif (current) değilse anında silinirler**. Ancak bir thread üzerinde o an render işlemi gerçekleştiriliyorsa ("current" ise), yüzey tipi gözetilmeksizin "pending destruction" sürecine girerler. Çift tamponlu sistemlerde page-flip mekanizması yarıda kesilmez, takas işleminin güvenle bitmesi beklenir.

Ayrıntılar ve Yaşam Döngüsü

Thread Güvenliği (Thread Safety) ve Pending Destruction: EGL 1.0

spesifikasyonuna göre `eglTerminate` thread-safe bir fonksiyondur. Ancak, kritik kural şudur: Bir EGLContext **aynı anda sadece bir thread’de** current olabilir. Eğer diğer bir thread aktif olarak bir EGLContext ‘i current yapmışsa ve render işlemine devam ediyorsa, main thread üzerinden `eglTerminate` çağrıldığında bu context (ve ona bağlı surface’ler) anında yok edilip uygulama çökertilmez. Bunun yerine nesneler **“Pending Destruction” (Bekleyen Yıkım)** statüsüne alınır. İlgili thread `eglMakeCurrent` ile context bağlantısını koparana kadar render edilebilir durumda kalırlar, fakat o display için yeni kaynak (yeni bir surface veya context) oluşturulmasına izin verilmez.

Eşzamanlama (Synchronization): `eglTerminate` çağrılmadan önce bekleyen tüm render işlemlerinin bitmiş olduğundan emin olmak, donanım asenkronizasyonundan kaynaklı race condition veya native obje hatası almamak için şarttır. Bu eşzamanlamayı (synchronization) sağlamak için:

1. `eglWaitGL()` : Eğer OpenGL(ES) kullanılıyorsa, kuyruktaki komutların grafik donanımında tamamen yürütülmesini bekler (bir nevi `glFinish` muadilidir).
2. `eglWaitNative()` : Eğer 2D native grafik API’leri ile aynı anda EGL üzerinden yüzey çizimi yapıldıysa, native sistemin (örneğin X11 veya DRM’nin) işlemlerini bitirmesi için beklenir.

Hata Matrisi

EGL 1.0 spesifikasyonunun “fonksiyon başarısız olduğunda hiçbir yan etki (side effect) bırakmamalıdır” kuralı gereği, hatalı çağrılarda state machine’de hiçbir değişiklik olmaz, kaynaklar varlığını sürdürür.

Durum	Sonuç (EGL Hata Kodu)	Yan Etkiler
<code>dpy</code> geçerli bir görüntü (display) değilse veya <code>EGL_NO_DISPLAY</code> ise	<code>EGL_BAD_DISPLAY</code>	Hiçbir kaynak silinmez, display’in state machine’i değişmez. Hata bayrağı set edilir.
<code>dpy</code> zaten sonlandırılmış (uninitialized) ise	Hata Yok (<code>EGL_SUCCESS</code>)	EGL 1.0’da uninitialized bir display’i terminate etmek geçerli kabul edilir ve hata dönmaz.

*[!WARNING] EGLDisplay “uninitialized” duruma geçtikten sonra, aynı display kullanılarak yapılacak `eglInitialize` , `eglMakeCurrent` ve `eglTerminate` dışındaki tüm EGL fonksiyon çağrıları (örneğin `eglChooseConfig` , `eglCreateContext`) anında **EGL_NOT_INITIALIZED** hatası fırlatır.*

Güvenli Kullanım Örneği

Aşağıdaki örnekte EGL 1.0 spesifikasyonlarına tamamen uygun, pending destruction bırakmadan (güvenli bir şekilde context ve yüzeyleri ayırarak) ve bellek sızıntısını engelleyerek programdan çıkan, hata kontrollerini (error checking) barındıran temiz bir C kodu kullanımı sunulmuştur:

```
#include <EGL/egl.h>
#include <GLES/gl.h>
#include <stdio.h>
#include <stdlib.h>

void safe_egl_cleanup(EGLDisplay dpy) {
    if (dpy == EGL_NO_DISPLAY) {
        printf("HATA: Geçersiz EGL Display (EGL_NO_DISPLAY).\n");
        return;
    }

    // 1. Eşzamanlama (Synchronization) - Donanım kuyruğunun bitmesi
    // OpenGL komutlarının tamamlandığından donanım seviyesinde em
    eglWaitGL();

    // Opsiyonel: Native rendering işlemlerinin EGL'yi beklemesini
    // eglWaitNative(EGL_CORE_NATIVE_ENGINE);

    // 2. Mevcut thread üzerindeki tüm context ve surface bağlarını
    // Bu sayede eglTerminate nesneleri "pending destruction" yerin
    if (eglMakeCurrent(dpy, EGL_NO_SURFACE, EGL_NO_SURFACE, EGL_NO
        printf("Uyarı: Context bağlantıları çözülemedi! Hata Kodu:
    }

    // 3. EGLDisplay bağlantısını sonlandır (eglTerminate)
    EGLBoolean result = eglTerminate(dpy);

    if (result == EGL_FALSE) {
        EGLint err = eglGetError();
        printf("KRİTİK HATA: eglTerminate başarısız oldu.\n");
        if (err == EGL_BAD_DISPLAY) {
            printf("Sebep: EGL_BAD_DISPLAY (Geçersiz veya bozuk EGL
        }
    } else {
        printf("BAŞARILI: EGL kaynakları ve oturum sorunsuz bir şel
    }

    // 4. Native kaynakların manuel olarak temizlenmesi
    // eglTerminate'den sonra GBM/DRM nesneleri serbest bırakılır.
    // gbm_device_destroy(gbm_dev);
    // close(drm_fd);
}
```

```
int main(void) {
    // EGL Başlatma Senaryosu (Örnek)
    EGLDisplay dpy = eglGetDisplay(EGL_DEFAULT_DISPLAY);

    if (dpy != EGL_NO_DISPLAY) {
        if (eglInitialize(dpy, NULL, NULL) == EGL_TRUE) {
            // ... Yüzey oluşturma, Context Atama, Render döngüsü

            // Program kapatılırken veya EGL'ye ihtiyaç bittiğinde
            safe_egl_cleanup(dpy);
        }
    }
    return 0;
}
```

Pratik Özet

- **Oturum Kapatıcı Sorumluluğu:** `eglTerminate` , `eglInitialize` işleminin zıttı olup uygulamadan çıkarken mutlaka çağrılmalıdır. Aksi halde EGL sürücüsü seviyesinde ciddi bellek sızıntıları (memory leak) yaşanır.
- **Pending Destruction (Bekleyen Yıkım):** Başka bir thread'de o an aktif (`current`) olan nesneler anında silinmez. Uygulamanın çökmesini engellemek için `eglMakeCurrent` ile serbest bırakılana kadar hafızada kalır. En iyi pratik, `eglTerminate` öncesinde `eglMakeCurrent(dpy, EGL_NO_SURFACE, EGL_NO_SURFACE, EGL_NO_CONTEXT)` çağırmasıdır.
- **Güvenli Uninitialized Davranışı (Idempotent):** EGL 1.0 kurallarına göre `eglTerminate` , halihazırda başlatılmamış (`uninitialized`) bir `dpy` parametresiyle çağrılrsa dahi programı bozmaz, `EGL_TRUE` döner ve hata üretmez. (Senaryo C)
- **Tek Yan Etki Hilesi:** Sadece `EGL_NO_DISPLAY` (veya bozuk/rastgele pointer) verilmesi durumunda side effect yaratmadan `EGL_FALSE` döner ve sisteme sadece `EGL_BAD_DISPLAY` hatası kaydeder. (Senaryo B)
- **State Machine İzolasyonu:** İşlem başarıyla sonuçlandıktan sonra `EGLDisplay` “uninitialized” duruma geçer. Bu durumdayken display üzerinden yeni bağlam veya yüzey (örn. `eglCreateContext`) oluşturulmaya kalkılırsa anında `EGL_NOT_INITIALIZED` hatası alınır.
- **Donanımsal Ayırma (DRM/KMS):** `eglTerminate` sadece EGL objelerinin temizlenmesinden sorumludur. Native yapınızdaki pencere sistemini (X11 Display) veya Kernel Mode Setting dosya tanımlayıcılarını (DRM fd, GBM Buffer) native API'ler (`close` , `gbm_surface_destroy`) kullanarak yok etmeniz gerekir.

EGL 1.0: `eglGetError`

```
EGLint eglGetError(void);
```

`eglGetError` , çağıran thread üzerinde en son EGL çağrısıyla ilişkili EGL hata durumunu döndürür.

Fonksiyon parametre almaz. Bunun nedeni hata bilgisinin belirli bir `EGLDisplay` , `EGLContext` veya `EGLSurface` handle'ı üzerinden değil, çağıran thread'in EGL hata durumu üzerinden okunmasıdır.

Mental Model

```
Thread
|
+-- EGL fonksiyon çağrısı
|   |
|   +-- başarılı / başarısız
|   |
|   v
|   error state
|
+-- eglGetError()
    |
    v
    EGLint hata kodu
```

Örnek:

```
EGLSurface surface =
    eglCreateWindowSurface(
        egl_display,
        egl_config,
        (EGLNativeWindowType)gbm_surface,
        NULL
    );

if (surface == EGL_NO_SURFACE) {
    EGLint err = eglGetError();
}
```

Parametreler

`eglGetError` parametre almaz:

```
eglGetError();
```

Dolayısıyla bu fonksiyonda diğer EGL fonksiyonlarındaki gibi:

```
dpy  
config  
surface  
context  
attribute
```

parametreleri yoktur.

İncelenecek ana konu fonksiyonun döndürebileceği `EGLint` değerleridir.

Dönüş Değeri

Fonksiyonun dönüş tipi:

```
EGLint
```

EGL 1.0'da dönebilecek hata kodları:

Değer	Anlamı
<code>EGL_SUCCESS</code>	Son ilgili EGL işlemi başarılıdır / hata yoktur.
<code>EGL_NOT_INITIALIZED</code>	EGL ilgili display için initialize edilmemiştir veya initialize edilememiştir.
<code>EGL_BAD_ACCESS</code>	İstenen kaynağa erişilememiştir.
<code>EGL_BAD_ALLOC</code>	İstenen işlem için gerekli kaynak ayrılamamıştır.
<code>EGL_BAD_ATTRIBUTE</code>	Tanınmayan attribute veya attribute değeri kullanılmıştır.
<code>EGL_BAD_CONTEXT</code>	Bir <code>EGLContext</code> argümanı geçerli context değildir.
<code>EGL_BAD_CONFIG</code>	Bir <code>EGLConfig</code> argümanı geçerli config değildir.

Değer	Anlamı
EGL_BAD_CURRENT_SURFACE	Thread'in current surface'i artık geçerli değildir.
EGL_BAD_DISPLAY	Bir <code>EGLDisplay</code> argümanı geçerli değildir veya display initialize edilmemiştir.
EGL_BAD_SURFACE	Bir <code>EGLSurface</code> argümanı geçerli surface değildir.
EGL_BAD_MATCH	Fonksiyon argümanları birbirleriyle uyumlu değildir.
EGL_BAD_PARAMETER	Bir veya daha fazla argüman değeri geçersizdir.
EGL_BAD_NATIVE_PIXMAP	Native pixmap geçerli değildir.
EGL_BAD_NATIVE_WINDOW	Native window geçerli değildir.

EGL_SUCCESS

```

EGLint err = eglGetError();

if (err == EGL_SUCCESS) {
    /* EGL hata durumu yok */
}

```

Anlamı:

```

EGL_SUCCESS
|
v
Fonksiyon başarılı / kayıtlı EGL hatası yok

```

EGL_NOT_INITIALIZED

Örneğin ilgili `EGLDisplay` initialize edilmeden EGL işlemi yapılmaya çalışılırsa görülebilir.

Doğru temel sıra:

```
EGLDisplay dpy = /* display'i elde et */;

EGLint major;
EGLint minor;

if (!eglInitialize(dpy, &major, &minor)) {
    EGLint err = eglGetError();

    if (err == EGL_NOT_INITIALIZED) {
        /* Initialize başarısız */
    }
}
```

Bu projede EGL display GBM platformuyla ilişkili olduğundan GBM/EGL platform kurulumunun başarılı olması gerekir.

EGL_BAD_ACCESS

Bir EGL kaynağına erişim kuralları ihlal edildiğinde dönebilir.

Örneğin bir context başka thread üzerinde current ise:

```
Thread A
|
+-- ctx current

Thread B
|
+-- aynı ctx'yi current yapmaya çalışır
    |
    v
    EGL_BAD_ACCESS
```

Örnek kontrol:

```
if (!eglMakeCurrent(dpy, surface, surface, ctx)) {
    EGLint err = eglGetError();

    if (err == EGL_BAD_ACCESS) {
        /* Kaynak başka thread tarafından kullanılıyor olabilir */
    }
}
```


EGL_BAD_ALLOC

EGL istenen işlem için yeterli kaynak ayıramazsa döner.

Örneğin:

```
EGLSurface surface =
    eglCreateWindowSurface(
        dpy,
        config,
        (EGLNativeWindowType)gbm_surface,
        NULL
    );

if (surface == EGL_NO_SURFACE) {
    EGLint err = eglGetError();

    if (err == EGL_BAD_ALLOC) {
        /* Surface için gerekli kaynak ayırlamadı */
    }
}
```

Bu, GBM fonksiyonlarının kendi hata modelinden ayrıdır. `eglGetError()` yalnızca EGL hata durumunu verir.

EGL_BAD_ATTRIBUTE

EGL attribute listesinde geçersiz attribute veya değer kullanıldığında oluşabilir.

Örnek genel EGL modeli:

```
const EGLint attribs[] = {
    /* EGL 1.0 tarafından desteklenmeyen/geçersiz bir attribute */
    0x12345678, 1,
    EGL_NONE
};
```

İlgili EGL fonksiyonu başarısız olduktan sonra:

```
EGLint err = eglGetError();

if (err == EGL_BAD_ATTRIBUTE) {
    /* Attribute listesi geçersiz */
}
```

EGL_BAD_CONTEXT

Bir EGL fonksiyonuna geçerli olmayan `EGLContext` verildiğinde oluşur.

Örneğin `eglDestroyContext` :

```
if (!eglDestroyContext(dpy, invalid_context)) {
    EGLint err = eglGetError();

    if (err == EGL_BAD_CONTEXT) {
        /* ctx geçerli EGLContext değil */
    }
}
```

EGL_BAD_CONFIG

Geçersiz `EGLConfig` handle'ı kullanıldığında oluşur.

Örneğin:

```
EGLSurface surface =
    eglCreateWindowSurface(
        dpy,
        invalid_config,
        (EGLNativeWindowType)gbm_surface,
        NULL
    );

if (surface == EGL_NO_SURFACE) {
    EGLint err = eglGetError();

    if (err == EGL_BAD_CONFIG) {
        /* config geçersiz */
    }
}
```

EGL_BAD_CURRENT_SURFACE

Thread üzerinde current olan surface artık geçerli olmadığında bazı EGL işlemlerinde görülebilir.

Mental model:

```

Thread
|
+-- current EGLSurface
    |
    X native/current surface geçersiz hale geldi
    |
    v
EGL_BAD_CURRENT_SURFACE

```

EGL_BAD_DISPLAY

Geçersiz veya uygun şekilde initialized edilmemiş display ile işlem yapılırsa görülebilir.

Örneğin:

```

if (!eglInitialize(dpy, &major, &minor)) {
    EGLint err = eglGetError();

    if (err == EGL_BAD_DISPLAY) {
        /* dpy geçerli EGLDisplay değil */
    }
}

```

EGL_BAD_SURFACE

Bir EGL fonksiyonuna geçerli olmayan surface verilirse oluşur.

Bu grubunuzdaki `eglDestroySurface` için doğrudan örnek:

```

if (!eglDestroySurface(dpy, invalid_surface)) {
    EGLint err = eglGetError();

    if (err == EGL_BAD_SURFACE) {
        /* surface geçerli değil */
    }
}

```

EGL_BAD_MATCH

Argümanlar tek tek geçerli olsa bile birbirleriyle uyumsuz olduğunda oluşabilir.

Örneğin:

```
EGLContext geçerli
EGLSurface geçerli
    |
    v
ancak birbirleriyle uyumsuz
    |
    v
EGL_BAD_MATCH
```

`eglMakeCurrent` çağrısında context ile surface uyumsuzsa:

```
if (!eglMakeCurrent(dpy, surface, surface, ctx)) {
    EGLint err = eglGetError();

    if (err == EGL_BAD_MATCH) {
        /* context / surface eşleşmesi uygun değil */
    }
}
```

Bu projede `eglCreateWindowSurface` sırasında native GBM surface ile EGLConfig uyumsuzluğu da `EGL_BAD_MATCH` üretebilir.

EGL_BAD_PARAMETER

Bir veya daha fazla parametre değeri ilgili fonksiyon açısından geçersiz olduğunda kullanılır.

```
EGLint err = eglGetError();

if (err == EGL_BAD_PARAMETER) {
    /* İlgili EGL fonksiyonunda parametre değeri geçersiz */
}
```

Hangi fonksiyonun hangi durumda bu hatayı ürettiği o fonksiyonun EGL 1.0 tanımına göre değerlendirilmelidir.

EGL_BAD_NATIVE_PIXMAP

Geçerli olmayan bir native pixmap handle'ı kullanıldığında oluşur.

Bu direct-to-display GBM projesinde native pixmap kullanılmadığından normal akışta beklenen bir hata değildir.

```
Bu proje:  
GBM native window -> kullanılıyor  
Native pixmap      -> kullanılmıyor
```

EGL_BAD_NATIVE_WINDOW

`NativeWindowType` geçerli native window'u temsil etmiyorsa oluşur.

Bu projede `eglCreateWindowSurface` çağrısında native window:

```
(EGLNativeWindowType)gbm_surface
```

olduğu için GBM surface native window rolündedir.

Örneğin:

```
EGLSurface surface =  
    eglCreateWindowSurface(  
        dpy,  
        config,  
        (EGLNativeWindowType)gbm_surface,  
        NULL  
    );  
  
if (surface == EGL_NO_SURFACE) {  
    EGLint err = eglGetError();  
  
    if (err == EGL_BAD_NATIVE_WINDOW) {  
        /* Native window geçerli değil */  
    }  
}
```

Bu Projede Kullanım Modeli

En faydalı kullanım, her EGL fonksiyonunun dönüş değerini önce kontrol edip sadece başarısız durumda `eglGetError()` çağırmasıdır.

Örneğin initialize:

```
if (!eglInitialize(egl_display, &major, &minor)) {  
    EGLint err = eglGetError();  
}
```

Surface oluşturma:

```
EGLSurface egl_surface =
    eglCreateWindowSurface(
        egl_display,
        egl_config,
        (EGLNativeWindowType)gbm_surface,
        NULL
    );

if (egl_surface == EGL_NO_SURFACE) {
    EGLint err = eglGetError();
}
```

Context oluşturma:

```
EGLContext egl_context =
    eglCreateContext(
        egl_display,
        egl_config,
        EGL_NO_CONTEXT,
        NULL
    );

if (egl_context == EGL_NO_CONTEXT) {
    EGLint err = eglGetError();
}
```

Make current:

```
if (!eglMakeCurrent(
    egl_display,
    egl_surface,
    egl_surface,
    egl_context)) {
    EGLint err = eglGetError();
}
```

Swap:

```
if (!eglSwapBuffers(  
    egl_display,  
    egl_surface)) {  
    EGLint err = eglGetError();  
}
```

Hata Yazdırma Yardımcı Fonksiyonu

Projede okunabilirlik için küçük bir yardımcı fonksiyon yazılabilir:

```
const char *egl_error_string(EGLint error)
{
    switch (error) {
        case EGL_SUCCESS:
            return "EGL_SUCCESS";

        case EGL_NOT_INITIALIZED:
            return "EGL_NOT_INITIALIZED";

        case EGL_BAD_ACCESS:
            return "EGL_BAD_ACCESS";

        case EGL_BAD_ALLOC:
            return "EGL_BAD_ALLOC";

        case EGL_BAD_ATTRIBUTE:
            return "EGL_BAD_ATTRIBUTE";

        case EGL_BAD_CONTEXT:
            return "EGL_BAD_CONTEXT";

        case EGL_BAD_CONFIG:
            return "EGL_BAD_CONFIG";

        case EGL_BAD_CURRENT_SURFACE:
            return "EGL_BAD_CURRENT_SURFACE";

        case EGL_BAD_DISPLAY:
            return "EGL_BAD_DISPLAY";

        case EGL_BAD_SURFACE:
            return "EGL_BAD_SURFACE";

        case EGL_BAD_MATCH:
            return "EGL_BAD_MATCH";

        case EGL_BAD_PARAMETER:
            return "EGL_BAD_PARAMETER";

        case EGL_BAD_NATIVE_PIXMAP:
            return "EGL_BAD_NATIVE_PIXMAP";

        case EGL_BAD_NATIVE_WINDOW:
```



```
        return "EGL_BAD_NATIVE_WINDOW";

    default:
        return "UNKNOWN_EGL_ERROR";
    }
}
```

Kullanım:

```
EGLint err = eglGetError();

printf(
    "EGL error: %s (0x%x)\n",
    egl_error_string(err),
    err
);
```

GBM / DRM Hatalarından Farkı

Bu proje üç ayrı API ailesi içerir:

```
OpenGL ES / EGL
GBM
DRM/KMS
```

`eglGetError()` sadece:

```
EGL hata state
```

bilgisini verir.

Şunların hata sistemini okumaz:

```
gbm_surface_create()
drmModeGetResources()
drmModeAddFB()
drmModeSetCrtc()
drmModePageFlip()
open("/dev/dri/card*")
```

DRM/Linux hatalarında genellikle fonksiyon dönüş değeri ve gerektiğinde `errno` kullanılır.

Yani:

```
EGL fonksiyonu başarısız -> eglGetError()  
DRM/Linux çağrısı      -> return value / errno  
GBM çağrısı            -> GBM fonksiyonunun dönüş kontrolü
```

Minimal Doğru Kullanım

```
if (!eglMakeCurrent(  
    egl_display,  
    egl_surface,  
    egl_surface,  
    egl_context)) {  
  
    EGLint err = eglGetError();  
  
    printf(  
        "eglMakeCurrent failed: 0x%x\n",  
        err  
    );  
}
```

Farklı Dönüş Değerlerine Göre Örnekler

1. EGL_SUCCESS

```
EGLint err = eglGetError();  
  
if (err == EGL_SUCCESS) {  
    printf("EGL error yok\n");  
}
```

2. EGL_BAD_CONTEXT

```
if (!eglDestroyContext(dpy, invalid_ctx)) {  
    EGLint err = eglGetError();  
  
    if (err == EGL_BAD_CONTEXT) {  
        printf("Context gecersiz\n");  
    }  
}
```

3. EGL_BAD_SURFACE

```
if (!eglDestroySurface(dpy, invalid_surface)) {
    EGLint err = eglGetError();

    if (err == EGL_BAD_SURFACE) {
        printf("Surface gecersiz\n");
    }
}
```

4. EGL_BAD_NATIVE_WINDOW

```
EGLSurface surface =
    eglCreateWindowSurface(
        dpy,
        config,
        native_window,
        NULL
    );

if (surface == EGL_NO_SURFACE) {
    EGLint err = eglGetError();

    if (err == EGL_BAD_NATIVE_WINDOW) {
        printf("Native window gecersiz\n");
    }
}
```

EGL 1.0 İçin Pratik Özet

- `eglGetError()` parametre almaz.
- Dönüş tipi `EGLint` 'tir.
- Thread'in EGL hata durumunu okumak için kullanılır.
- `EGL_SUCCESS` , EGL hata durumu olmadığını belirtir.
- EGL 1.0 tüm temel EGL hata kodlarını bu fonksiyon üzerinden raporlar.
- En doğru kullanım, önce ilgili EGL fonksiyonunun başarısız olup olmadığını kontrol etmek ve ardından `eglGetError()` çağırmasıdır.
- GBM veya DRM/KMS hataları `eglGetError()` ile alınmaz.
- Direct-to-display projede EGL, GBM ve DRM/KMS hata kontrolleri birbirinden ayrı tutulmalıdır.

